

Petal Form Dual-Axis Solar Tracker

Software-in-the-Loop (SIL) ve Dijital İkiz Geliştirme Planı
MATLAB / Simulink Tabanlı Uygulama Kılavuzu

Sistem Doğrulama Dokümanı — v1.0

Mayıs 2026

Doküman Kapsamı

Bu doküman, daha önce kararlaştırılmış olan “Petal Form Dual-Axis Solar Tracker” projesinin fiziksel olarak inşa edilmeden önce MATLAB/Simulink ortamında dijital ikiz (digital twin) olarak doğrulanmasına yönelik adım adım SIL kurulumunu detaylandırır.

Hedef, sahaya çıkmadan önce kontrol algoritması, durum makinesi, sensör akışı ve iki-MCU iletişim mimarisinin doğruluğunu kanıtlamaktır.

İlgili Doküman: *Petal Form Dual-Axis Solar Tracker — Tam Sistem Dokümanı v2.0*

Hedef Kullanıcı: Akademik prototip geliştirici (yüksek lisans / doktora)

Saha: Gran Canaria — yüksek ışınım test alanı

Contents

1 Giriş ve Doküman Amacı	7
1.1 Proje Bağlamı	7
1.2 Neden SIL? — Yapılabilirlik Gerekçesi	7
1.3 Dokümanın Yapısı	7
2 Sistem Mimarisi Özeti	8
2.1 Çift-MCU Yaklaşımı	8
2.2 Sensör Topolojisi	8
2.3 LDR Hata Hesabı	8
2.4 PI Hız Kontrolcüsü	8
2.5 UART Çerçeve Formatı	9
2.6 FSM Durumları	9
3 SIL Stratejisi: 3-Aşamalı Yaklaşım	9
3.1 Genel Felsefe	9
3.2 Aşamalar Arası Geçiş Kriterleri	9
3.3 Karar: Aşama 1 Tamamen MATLAB İçinde	10
4 Aşama 1: MATLAB Tabanlı Mantıksal İkiz	10
4.1 Genel Mimari	10
4.2 Simulink Model Hazırlığı	10
4.3 Plant_Subsystem — Mevcut Kodun Sarmalanması	11
4.4 STM32_Controller — Stateflow FSM ve Ayırık PI	12
4.4.1 Stateflow Diyagramı	13
4.4.2 PI Step Fonksiyonu (MATLAB Function blok içinde)	13
4.4.3 UART Frame Üretimi	14
4.5 ESP32_Logger — Sensör, UART Parse, CSV Yazma	14
4.6 UART İletişim Modellemesi	16
4.7 SD Blokaj ve Parazitik Tüketim Modeli	16
4.8 Çıktı: CSV Üretimi ve Enerji Doğrulaması	17
4.9 Aşama 1'in Çıktısı: Tek Akademik Grafik	17
5 Yardımcı Fonksiyon Kütüphanesi	17
5.1 Güneş Efemeris Modeli	18
5.2 LDR Septum Geometrik Modeli	20
5.3 PV Panel I-V Modeli (Tek-Diyot)	21
5.4 Faıman Termal Modeli	22
5.5 Parazitik Güç Bütçesi	22
5.6 Motor Dinamiği Modeli	24
5.7 UART Frame Parser (ESP32 Tarafında)	25
5.8 LiFePO4 Batarya Modeli	26
5.9 LDR Septum Kalibrasyon Protokolü	27
5.9.1 Kalibrasyon Düzenneği	27
5.9.2 Kalibrasyon Adımları	27
5.10 PVGIS Veri Yükleme Pipeline'ı	28

5.11	Ana Koşum Scripti	30
6	Doğrulama ve İstatistiksel Analiz Metodolojisi	32
6.1	Doğrulama Hiyerarşisi	32
6.2	Birim Testleri (MATLAB Unit Test Framework)	32
6.3	Stateflow Chart Kapsama Analizi	33
6.4	Hipotez Testi: Net Kazanç İstatistiksel Olarak Pozitif mi?	34
6.5	Performans Metrikleri (KPI'lar)	35
6.6	İstatistiksel Power Analizi: Kaç Koşum Gerekli?	35
6.6.1	Power Analizi Formülü	35
6.6.2	Bu Proje İçin Hesap	35
6.6.3	MATLAB ile Power Curves	36
6.6.4	Pilot Koşumdan σ Tahmini	37
6.6.5	Power Analizinin Yorumlanması	37
6.7	Performans Metrikleri (KPI'lar) ²	37
7	Parametre Duyarlılık Analizi (Monte Carlo)	38
7.1	Duyarlılık Aranan Parametreler	38
7.2	Parsim ile Paralel Tarama	38
7.3	Tornado Diyagramı Yorumlama	39
7.4	Pareto Optimal Konfigürasyonun Aranması	39
8	Görselleştirme ve Akademik Şekil Üretimi	40
8.1	Yayın Kalitesinde Grafik Ayarları	40
8.2	Renk-Körü Dostu Palet (Wong, 2011)	40
8.3	Yayın için Export Komutları	41
8.4	Standart Tez Figürleri	41
9	Aşama 2: Firmware Düzeyi SIL (Renode + Wokwi)	41
9.1	STM32 Tarafı: Renode	41
9.1.1	Kurulum	41
9.1.2	Renode Script (.resc)	42
9.1.3	ADC Inject Mekanizması (Kritik)	43
9.2	ESP32 Tarafı: Wokwi CLI	43
9.2.1	Kurulum	43
9.2.2	Wokwi Config (wokwi.toml)	43
9.3	Hardware Abstraction Layer (HAL) Tasarımı	43
9.3.1	HAL Arayüzü	44
9.3.2	Üç Implementasyon Stratejisi	44
9.3.3	Renode Backend Örneği	44
9.4	Renode Python Plugin: ADC Inject Mekanizması	45
9.4.1	MATLAB Tarafından Bağlanma	46
9.5	Wokwi Diagram Tanımı	47
9.6	Stateflow → Embedded Coder: Otomatik C Üretimi	48
9.6.1	Embedded Coder Hazırlığı	48
9.6.2	Üretilen Dosyalar	48

9.6.3	STM32CubeIDE'ye Entegrasyon	48
9.6.4	Eşdeğerlik (Equivalence) Testi	49
9.7	Aşama 2 Adım Adım Çalışma Akışı	50
9.8	Aşama 2'de Test Edilecek Yeni Şeyler	50
10	Aşama 3: Tam Co-Simulation	51
10.1	Port Tahsisi	51
10.2	MATLAB Simulink TCP Blokları	51
10.3	Co-Simulation Döngüsü	51
10.4	Aşama 3 Çıktısı: Üçlü Karşılaştırma Grafiği	52
11	Test Senaryoları	52
12	Yazılım Araçları ve Kurulum	53
13	Zaman Çizelgesi	54
14	CI/CD: Sürekli Entegrasyon ve Regresyon Testleri	54
14.1	Neden CI/CD?	54
14.2	GitHub Actions Workflow	55
14.3	Baseline Karşılaştırması	56
14.4	Baseline Güncelleme Politikası	57
14.5	Bildirim ve Görselleştirme	57
15	Donanım Bring-up ve Faz 6 Detayı	57
15.1	Pre-Installation Doğrulama Kontrol Listesi	58
15.2	Mekanik Montaj Sırası	58
15.3	Elektriksel Bağlantı Sırası — Asla Tek Adımda Tüm Sistemi Bağlama!	58
15.4	First Boot Doğrulaması	59
15.5	On-Site LDR Kalibrasyonu (Bölüm 5 ile bağlantı)	59
15.6	Veri Toplama Doğrulama Periyodu	60
15.7	Saha Hata Mod ve Etki Analizi (FMEA)	60
15.8	SIL→Saha Çapraz Referans	60
16	Sorun Giderme Rehberi (Troubleshooting)	60
16.1	Aşama 1 (MATLAB / Simulink) Sorunları	60
16.1.1	“Algebraic loop detected”	60
16.1.2	“Stateflow chart yields wrong output type”	61
16.1.3	“FSM hiç MOVE durumuna girmiyor”	61
16.1.4	“parsim çok yavaş”	61
16.2	Aşama 2 (Renode + Wokwi) Sorunları	62
16.2.1	“Renode .elf yükleyemiyor”	62
16.2.2	“Wokwi UART verisi gelmiyor”	62
16.2.3	“ADC bridge plugin yüklenmedi”	62
16.3	Aşama 3 (Co-Simulation) Sorunları	62
16.3.1	“Senkronizasyon bozuk: MATLAB çok hızlı”	62
16.3.2	“TCP retry storm”	62

16.3.3	“Co-simulation’da E_net Aşama 1’den çok düşük”	63
16.4	Donanım/Saha Geçişi Sorunları (Önleyici)	63
16.4.1	“INA226 sapması Aşama 1’le tutmuyor”	63
16.4.2	“Saha akşam: petal kapanmıyor”	63
16.5	Genel Debug Stratejisi: “Divide-and-Conquer”	63
17	Akademik Yazım ve Veri Sunumu	64
17.1	Tez Bölümü Önerilen Yapısı	64
17.2	Numerik Sonuçların Sunumu	64
17.3	Şekil Yazım Standardı	64
17.4	Tablo Yazım Standardı	65
17.5	Reproducibility Checklist	65
17.6	Hipotez Yanlış Çıkarsa Ne Yazarsın?	65
18	Akademik Değer ve Sonraki Adımlar	66
18.1	Tez/Makalede SIL’in Yeri	66
18.2	Bu Plana Bağlı Yan Çalışmalar	66
18.3	Riskler ve Azaltmaları	66
18.4	Sonraki Adım (Bu Doküman Sonrası)	66
A	Ek A: Tam Plant_Subsystem MATLAB Function Şablonu	67
B	Ek B: Stateflow Chart Kurulumu Adımları	68
C	Ek C: Tam Stateflow Chart İçeriği	68
C.1	Chart Properties	69
C.2	Chart Inputs/Outputs/Locals	69
C.3	State: INIT	69
C.4	State: SLEEP	70
C.5	State: WAKE	70
C.6	State: DECIDE	71
C.7	State: MOVE	71
C.8	State: REPORT	72
C.9	State: SAFE	72
D	Ek D: Post-Process Boru Hattı	72
E	Ek E: Renode Dosya Şablonları	74
E.1	solar_tracker.resc Tam İçerik	74
E.2	Renode Konsol Üzerinden Manuel ADC Set	75
E.3	STM32G431KB Platform Açıklaması (.repl)	75
E.4	Renode’da Debug Komutları	77
E.5	GDB ile Birleşik Debugging	77
F	Ek F: Wokwi Komut Satırı Kullanımı	78
F.1	Komut Satırı Çalıştırma	78
F.2	Wokwi’den UART Loglarını Çekme	78

G Ek G: Test Raporu Şablonu	78
H Ek H: Sözlük (Glossary)	79

1 Giriş ve Doküman Amacı

1.1 Proje Bağlamı

Petal Form Dual-Axis Solar Tracker, küçük ölçekli (<50 W) güneş takip sistemlerinde *net enerji kazancı hipotezini* test eden bir akademik prototiptir. Sistem, çift mikrodenetleyici mimarisi (STM32 G431KB hareket kontrolü için, ESP32 WROOM-32 veri kaydı için) üzerine kurulmuş; “move-and-sleep” felsefesiyle parazitik tüketimi minimize etmeyi hedeflemektedir. Mevcut sistem dokümanı (v2.0) donanım, devre şemaları, durum makinesi (FSM), UART protokolü ve 7-fazlı inşaa planını detaylı olarak tanımlamaktadır. MATLAB tarafında ise tüm teorik kontrol algoritması, güneş efemerisi, Faiman termal model ve PI hız kontrolcüsü zaten yazılmış durumdadır.

1.2 Neden SIL? — Yapılabilirlik Gerekçesi

Tablo 1 projenin temel hipotezinin ne kadar dar bir marjla işlediğini gösterir:

Table 1: Yapılabilirlik analizinden enerji marjı özeti (Gran Canaria, yaz ayları).

Senaryo	İyimser	Gerçekçi	Kötümser
Takip ek üretimi (Wh/gün)	21.0	12.5	5.25
Parazitik tüketim (Wh/gün)	7.0	10.0	14.0
Net kazanç (Wh/gün)	+14.0	+2.5	−8.75

Gerçekçi senaryoda net kazanç sadece +2.5 Wh’dır. INA226 kalibrasyonunda %5’lik bir hata veya FSM’deki bir tek yanlış uyanma döngüsü bu marjı yutar. Dolayısıyla:

- Donanımı satın aldıktan ve haftalarca laboratuvar zamanı harcadıktan sonra “hipotez yanlış çıkıyor” demek istemiyoruz.
- Hipotezin yanlış olması bile akademik olarak değerli; ancak yanlışlığın nedeni *algoritma hatası* değil, *fizik gerçeği* olmalıdır.
- SIL, algoritmik hatayı fizik gerçeğinden ayırmanın tek yoludur.

1.3 Dokümanın Yapısı

- **Bölüm 2** sistemin orijinal mimarisini özetler.
- **Bölüm 3** 3-aşamalı SIL stratejisini tanımlar.
- **Bölüm 4** Aşama 1 (MATLAB tabanlı mantıksal ikiz) *tam uygulama detayı* ile açıklanır — bu, ana çalışma yükünün olduğu bölümdür.
- **Bölüm 9** ve **Bölüm 10** ileri aşamaları (firmware düzeyi SIL ve co-simulation) genel hatlarıyla ele alır.
- **Bölüm 11** test senaryolarını listeler.
- **Bölüm 13** 4 haftalık iş planını verir.

2 Sistem Mimarisi Özeti

Tam mimari detayı orijinal sistem dokümanında bulunduğu için burada sadece SIL çalışması için gerekli olan tanımlar tekrar edilmiştir.

2.1 Çift-MCU Yaklaşımı

Sistemin iki ayrı işlemciye bölünmesinin tek nedeni FAT32 dosya sistemi güncellemelerinden kaynaklanan **10–50 ms blokaj** süreleridir. Tek MCU’lu mimaride bu süre boyunca PI kontrol döngüsü çalışmaz ve motor hedefi kaçıır.

- **STM32 G431KB:** Gerçek-zamanlı, deterministik. Sadece LDR oku, FSM kararı ver, motoru sür, uyu. Aktif duty cycle yaklaşık %17.
- **ESP32 WROOM-32:** Sürekli uyanık. Sensör oku, zaman damgala, SD karta yaz. Karmaşık ama zaman-kritik değil.

2.2 Sensör Topolojisi

- 4 panel $\times$ 5 W @ 6 V (paralel bağlı)
- 6 adet INA226 (4 panel başına + 1 MPPT çıkışı + 1 sistem bus)
- 4 adet DS18B20 (her panelin arkasında, Faiman termal doğrulaması için)
- 4 adet LDR (piramidal septum yapısında — güneş pozisyon hatası ölçümü)
- 1 adet DS3231 RTC (zaman damgası)
- 1 adet microSD modülü (CSV log)

2.3 LDR Hata Hesabı

Septum geometrisi ile bölünmüş 4 LDR’nin ADC okumalarından pan ve tilt hataları:

$$e_{\text{pan}} = (\text{LDR}_1 + \text{LDR}_3) - (\text{LDR}_2 + \text{LDR}_4) \quad (1)$$

$$e_{\text{tilt}} = (\text{LDR}_1 + \text{LDR}_2) - (\text{LDR}_3 + \text{LDR}_4) \quad (2)$$

Deadband (ölü bölge):

$$|e_{\text{pan}}| < \epsilon_{db} \quad \wedge \quad |e_{\text{tilt}}| < \epsilon_{db} \implies \text{Hareket yok, uykuya geç} \quad (3)$$

Tipik değer: $\epsilon_{db} \approx 100$ (12-bit ADC’de, $\approx 6^\circ$ ’ye karşılık).

2.4 PI Hız Kontrolcüsü

Ayrık zamanlı form (örnekleme periyodu T_s):

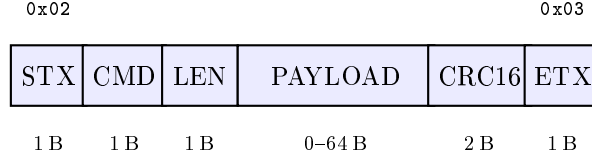
$$u[k] = K_p \cdot e[k] + K_i \cdot T_s \sum_{j=0}^k e[j] \quad (4)$$

Anti-windup:

$$I[k] = \text{clamp}(I[k-1] + e[k] \cdot T_s, -I_{\max}, +I_{\max}) \quad (5)$$

Başlangıç değerleri: $K_p = 0.5$, $K_i = 0.1$, $I_{\max} = 500$, $T_s = 50$ ms.

2.5 UART Çerçeve Formatı



Komut tipleri: 0x10 (durum raporu), 0x11 (alarm), 0x20 (zaman senk.), 0x21 (config), 0xAA (heartbeat), 0xFF (ACK).

2.6 FSM Durumları

7 durumlu sonlu durum makinesi: INIT, SLEEP, WAKE, DECIDE, MOVE, REPORT, SAFE. “Move-and-sleep” felsefesi gereği STM32 yalnızca WAKE-MOVE-REPORT adımları sırasında aktiftir; aksi durumda Stop Mode’da ~ 10 μ A çeker.

3 SIL Stratejisi: 3-Aşamalı Yaklaşım

3.1 Genel Felsefe

SIL çalışması artımsal sadakatle (incremental fidelity) ilerler. Bu, NASA’nın uzay araç doğrulamasında kullandığı yaklaşımın küçük-ölçek karşılığıdır. Her aşama bağımsız çalışan bir sistem üretir; bir sonraki aşama sadece gerçekçilik düzeyini artırır.

Table 2: 3-aşamalı SIL stratejisi: amaç, araç ve doğrulama düzeyi.

Aşama	Amaç	Ana Araç	Doğrulan Düzey
1	Algoritma ve veri akışı mantığı	MATLAB / Simulink / Stateflow	Kontrol algoritması, FSM, PI, UART protokol mantığı
2	Gömülü kod davranışı	Renode (STM32) + Wokwi (ESP32)	Gerçek C firmware, timing, ISR, watchdog
3	Tam co-simulation	MATLAB $\leftrightarrow$ Renode $\leftrightarrow$ Wokwi (TCP)	Üçlü doğrulama; sahaya çıkma kararı

3.2 Aşamalar Arası Geçiş Kriterleri

- **Aşama 1 $\rightarrow$ 2:** 24 saatlik sanal Gran Canaria simülasyonunda net enerji $> +2$ Wh, veri kaybı $< \%0.1$, sıfır CRC hatası, sıfır FSM kilitlenmesi.
- **Aşama 2 $\rightarrow$ 3:** STM32 firmware’i Renode’da 1 saatlik koşumda watchdog reset vermeden çalışır; ESP32 firmware’i Wokwi’de bellek sızıntısı göstermez.

- **Aşama 3 → Saha:** 30 günlük TMY verisi üzerinde co-simulation çalıştırıldığında net enerji teorik MIL değerinin %85'inden fazla bir oranı yakalar (firmware kaynaklı kayıp < %15).

3.3 Karar: Aşama 1 Tamamen MATLAB İçinde

Kullanıcının mevcut MATLAB kodu olduğu için, Aşama 1 **tek bir Simulink modeli içinde** üç alt sistem (subsystem) olarak kurulacaktır:

- **Plant_Subsystem:** Güneş, panel, motor, sensör fiziği.
- **STM32_Controller:** Stateflow FSM + ayrık PI.
- **ESP32_Logger:** Sensör örnekleme + UART parsing + CSV yazma + SD blokaj simülasyonu.

Bu yaklaşım iki ayrı MATLAB sürecine / TCP soketine ihtiyaç bırakmaz, mevcut kodu yeniden kullanır, ve tek Simulink Scope'unda tüm sinyalleri gözleme imkanı verir.

4 Aşama 1: MATLAB Tabanlı Mantıksal İkiz

Bu bölüm, dokümanın ana çalışma kılavuzudur. Tüm sonraki aşamalar bu yapının üzerine eklemelidir.

4.1 Genel Mimari

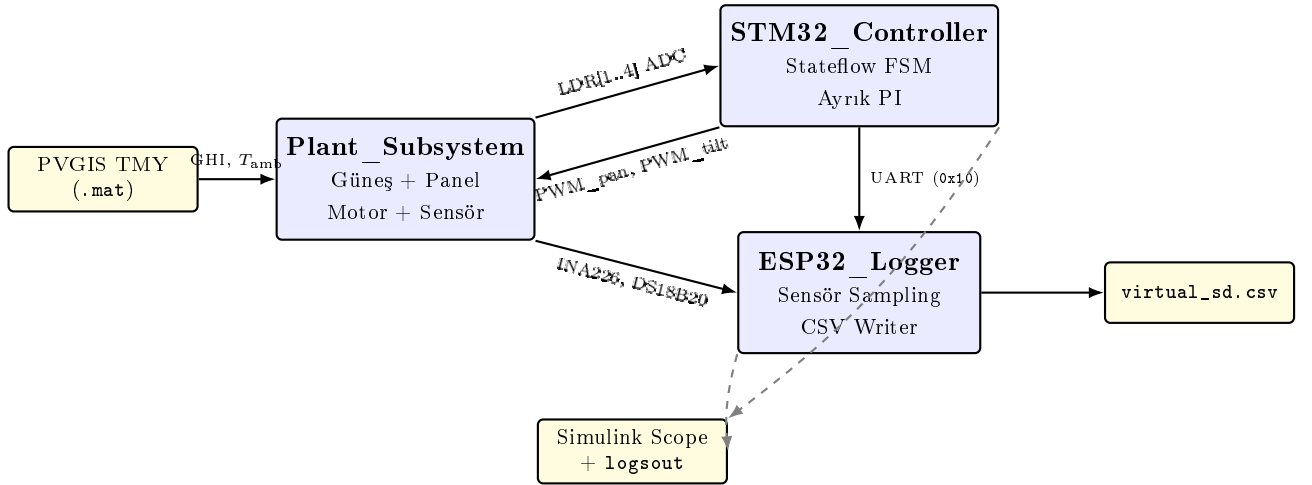


Figure 1: Aşama 1 Simulink mimarisi: tek model, üç alt sistem, TMY veri girişi, CSV çıkışı.

4.2 Simulink Model Hazırlığı

Yeni bir Simulink model oluştur (`solar_tracker_SIL.slx`). Aşağıdaki ayarları yap (Model Settings → Solver):

- **Type:** Fixed-step
- **Solver:** ode4 (Runge-Kutta)
- **Fundamental sample time:** 0.01 s (10 ms)

- **Stop time:** 86400 (24 saat, hızlandırılmış simülasyon)
- **Simulation pacing:** Açıkken $1\times$ (gerçek zamanlı), kapalıyken maksimum

Not: Hızlandırılmış simülasyon için *pacing*'i kapalı tut; 24 saat tipik bir laptop'ta yaklaşık 3–8 dakikada koşar. Görsel doğrulama gerektiğinde *pacing*'i $1\times$ yap.

4.3 Plant_Subsystem — Mevcut Kodun Sarmalanması

Mevcut MATLAB kodunda zaten güneş efemerisi, panel modeli, motor dinamikleri ve Faiman termal fonksiyonların var. Bu alt sistemin görevi onları Simulink imzasına ($u_k \rightarrow y_k$) sarmaktır.

Girişler:

- `pwm_pan` (int16, [-1000, +1000])
- `pwm_tilt` (int16, [-1000, +1000])
- `t_sim` (saniye, simulation time, otomatik mevcut)

Çıkışlar:

- `ldr_adc[4]` (uint16, 12-bit ADC değerleri, 0–4095)
- `P_panel[4]` (float, W)
- `V_panel[4]` (float, V)
- `I_panel[4]` (float, A)
- `P_mppt` (float, W — toplam)
- `P_bus` (float, W — parazitik)
- `T_panel[4]` (float, °C — Faiman çıktısı)
- `sun_az, sun_el` (float, ham gerçek — debug için)
- `panel_pan, panel_tilt` (float, gerçek panel açısı)

İç Yapı: Bir adet MATLAB Function bloğu yeterlidir. Aşağıdaki iskeleti mevcut kodunla birleştir:

```

1 function [ldr_adc, P_panel, V_panel, I_panel, P_mppt, P_bus, ...
2           T_panel, sun_az, sun_el, panel_pan_out, panel_tilt_out] = ...
3           plant_step(pwm_pan, pwm_tilt, t_sim, GHI, T_amb, wind)
4
5 %codegen
6 persistent panel_pan panel_tilt motor_pan_omega motor_tilt_omega
7 if isempty(panel_pan)
8     panel_pan = 0; panel_tilt = 0;
9     motor_pan_omega = 0; motor_tilt_omega = 0;
10 end
11
12 dt = 0.01; % fundamental step
13

```

```

14 % ---- 1) Güneş efemerisi (mevcut kodun) ----
15 [sun_az, sun_el] = solar_ephemeris_local(t_sim);
16
17 % ---- 2) Hizasizlik ve LDR voltajları ----
18 mis_pan = sun_az - panel_pan;
19 mis_tilt = sun_el - panel_tilt;
20 ldr_v = ldr_septum_model(mis_pan, mis_tilt, GHI); % 4x1, [V]
21 ldr_adc = uint16( min(4095, max(0, round(ldr_v * 4095 / 3.3))) );
22
23 % ---- 3) Panel optik ve termal modeli ----
24 [P_panel, V_panel, I_panel] = panel_iv_model(mis_pan, mis_tilt, GHI);
25 T_panel = faiman_temp(GHI, T_amb, wind); % [4 x 1]
26 P_mppt = sum(P_panel);
27 P_bus = parasitic_power_model(); % bkz. Bölüm 4.7
28
29 % ---- 4) Motor dinamiği (basit 2. derece) ----
30 [motor_pan_omega, panel_pan] = motor_step(double(pwm_pan), ...
31                                           motor_pan_omega, panel_pan, dt);
32 [motor_tilt_omega, panel_tilt] = motor_step(double(pwm_tilt), ...
33                                           motor_tilt_omega, panel_tilt, dt);
34
35 panel_pan_out = panel_pan;
36 panel_tilt_out = panel_tilt;
37 end

```

Listing 1: Plant_Subsystem MATLAB Function bloğu iskelet.

Motor adım modeli (önerilen — mevcut kodun varsa onu kullan):

$$J\ddot{\theta} + B\dot{\theta} = K_{\tau} \cdot u(t) - \tau_{\text{load}} \quad (6)$$

Worm gear'lı DC motor için tipik parametreler: $J = 0.005 \text{ kg} \cdot \text{m}^2$, $B = 0.05 \text{ N} \cdot \text{m} \cdot \text{s}/\text{rad}$, $K_{\tau} = 0.02 \text{ N} \cdot \text{m}/\%$ (%PWM cinsinden).

4.4 STM32_Controller — Stateflow FSM ve Ayrık PI

Bu alt sistem orijinal dokümandaki Şekil 5'in Simulink karşılığıdır. Bir **Triggered Subsystem** olarak modellenir; tetiklenme periyodu $T_s = 50 \text{ ms}$ olacak şekilde **Rate Transition** bloğu eklenir.

Girişler:

- ldr_adc[4] (Plant'tan)
- vbat (Plant'tan veya sabit)

Çıkışlar:

- pwm_pan, pwm_tilt
- fsm_state (1-7)
- uart_frame_out (16-byte vector — ESP32'ye gidecek payload)

- report_trigger (boolean — rapor zamanı geldi)

4.4.1 Stateflow Diyagramı

Stateflow Chart oluştur. 7 ana durum:

Listing 2: Stateflow durum geçiş yapısı (pseudocode).

```
INIT:
    entry: init_pi(); init_timers();
    on init_done: -> SLEEP;

SLEEP:
    entry: t_wake = t_sim + sleep_duration;
    during: if t_sim >= t_wake then -> WAKE;
    on emergency_alarm: -> SAFE;

WAKE:
    entry: ldr_snapshot = ldr_adc;
    -> DECIDE;

DECIDE:
    during:
        e_pan = (ldr[0]+ldr[2]) - (ldr[1]+ldr[3]);
        e_tilt = (ldr[0]+ldr[1]) - (ldr[2]+ldr[3]);
        if abs(e_pan) < deadband && abs(e_tilt) < deadband:
            -> REPORT;
        else:
            -> MOVE;

MOVE:
    during:
        [pwm_pan, integral_pan] = pi_step(e_pan, integral_pan);
        [pwm_tilt, integral_tilt] = pi_step(e_tilt, integral_tilt);
        move_timer += Ts;
    on (abs(e_pan)<deadband && abs(e_tilt)<deadband):
        pwm_pan = 0; pwm_tilt = 0; -> REPORT;
    on move_timer > 3.0:
        -> SAFE;    % stall guard

REPORT:
    entry: build_uart_frame(); report_trigger = 1;
    during: report_trigger = 0; -> SLEEP;

SAFE:
    entry: pwm_pan = 0; pwm_tilt = 0; log_event();
    on operator_reset: -> INIT;
```

4.4.2 PI Step Fonksiyonu (MATLAB Function blok içinde)

```
1 function [u, integral_out] = pi_step(e, integral_in, Kp, Ki, Ts, Imax, Umax)
2 %#codegen
3     integral_out = integral_in + e * Ts;
4     if integral_out > Imax, integral_out = Imax; end
5     if integral_out < -Imax, integral_out = -Imax; end
6     u_raw = Kp * e + Ki * integral_out;
```

```

7   u = max(-Umax, min(Umax, u_raw));
8   end

```

Listing 3: Anti-windup'lı ayırık PI adım fonksiyonu.

4.4.3 UART Frame Üretimi

REPORT durumunda entry eyleminde frame oluştur:

```

1  function frame = build_uart_frame(pan, tilt, ldr, state, petal, vbat)
2  %#codegen
3      STX = uint8(2); ETX = uint8(3); CMD = uint8(16);    % 0x10
4      payload = uint8(zeros(16,1));
5      payload(1:2) = typecast(int16(round(pan*10)), 'uint8');
6      payload(3:4) = typecast(int16(round(tilt*10)), 'uint8');
7      payload(5:6) = typecast(uint16(ldr(1)), 'uint8');
8      payload(7:8) = typecast(uint16(ldr(2)), 'uint8');
9      payload(9:10) = typecast(uint16(ldr(3)), 'uint8');
10     payload(11:12) = typecast(uint16(ldr(4)), 'uint8');
11     payload(13) = uint8(state);
12     payload(14) = uint8(petal);
13     payload(15:16) = typecast(uint16(round(vbat*10)), 'uint8');
14     LEN = uint8(length(payload));
15     body = [CMD; LEN; payload];
16     crc = crc16_ccitt([STX; body]);
17     frame = [STX; body; typecast(uint16(crc), 'uint8'); ETX];
18 end
19
20 function crc = crc16_ccitt(bytes)
21 %#codegen
22     crc = uint16(hex2dec('FFFF'));
23     for k = 1:length(bytes)
24         crc = bitxor(crc, bitshift(uint16(bytes(k)), 8));
25         for i = 1:8
26             if bitand(crc, uint16(hex2dec('8000'))) ~= 0
27                 crc = bitxor(bitshift(crc, 1), uint16(hex2dec('1021')));
28             else
29                 crc = bitshift(crc, 1);
30             end
31             crc = bitand(crc, uint16(hex2dec('FFFF')));
32         end
33     end
34 end

```

Listing 4: UART frame oluşturma (0x10 durum raporu).

4.5 ESP32_Logger — Sensör, UART Parse, CSV Yazma

Bu alt sistem 1 Hz periyotla çalışır. Aynı zamanda STM32'den gelen UART frame'ini parse eder ve SD kart blokağını simüle eder.

Girişler:

- P_panel[4], V_panel[4], I_panel[4] (INA226 #1-4)

- P_mppt (INA226 #5)
- P_bus (INA226 #6)
- T_panel[4] (DS18B20)
- uart_frame_in (STM32'den, 22-byte)
- report_trigger (yeni frame geldi)

Çıkışlar:

- csv_line_signal (Simulink To Workspace ile loglanır)
- sd_block_active (boolean — gözlem amaçlı)
- stm32_timeout (boolean — heartbeat alarmı)

```

1 function [csv_line, sd_block, stm32_timeout, free_heap] = ...
2     esp32_step(P_panel, V_panel, I_panel, P_mppt, P_bus, T_panel, ...
3         uart_frame, report_trigger, t_sim)
4 ##codegen
5 persistent last_stm32_hb sd_block_until heap_used
6 if isempty(last_stm32_hb)
7     last_stm32_hb = 0;
8     sd_block_until = 0;
9     heap_used = 0;
10 end
11
12 stm32_state = uint8(0); pan = single(0); tilt = single(0);
13 ldr_rep = uint16(zeros(4,1));
14
15 % ---- 1) UART frame parsing ----
16 if report_trigger > 0
17     [ok, parsed] = parse_uart_frame(uart_frame);
18     if ok
19         pan        = parsed.pan;
20         tilt        = parsed.tilt;
21         ldr_rep     = parsed.ldr;
22         stm32_state = parsed.state;
23         last_stm32_hb = t_sim;
24     else
25         % CRC fail --> log but proceed
26     end
27 end
28
29 % ---- 2) Heartbeat watchdog ----
30 stm32_timeout = (t_sim - last_stm32_hb) > 120; % 120s
31
32 % ---- 3) SD blokaj simülasyonu ----
33 % Her 1 Hz örnekte %5 olasilikla 10-50 ms blokaj
34 % (Plant_step her 10 ms cagrilidigi icin random tetikleme)
35 if (mod(t_sim,1) < 0.01) && (rand() < 0.05)
36     block_dur = 0.010 + 0.040*rand();
37     sd_block_until = t_sim + block_dur;
38 end

```

```

39 sd_block = boolean(t_sim < sd_block_until);
40
41 % ---- 4) CSV satiri olustur ----
42 % (Sadece her 1 saniyede ve blokaj DEGILse yaz)
43 csv_line = zeros(1, 24, 'single');
44 if (mod(t_sim, 1) < 0.01) && ~sd_block
45     csv_line(1) = single(t_sim);
46     csv_line(2) = single(pan);
47     csv_line(3) = single(tilt);
48     csv_line(4:7) = single(ldr_rep);
49     csv_line(8:11) = single(V_panel);
50     csv_line(12:15) = single(I_panel);
51     csv_line(16) = single(P_mppt);
52     csv_line(17) = single(P_bus);
53     csv_line(18:21) = single(T_panel);
54     csv_line(22) = single(stm32_state);
55     csv_line(23) = single(stm32_timeout);
56     csv_line(24) = single(sd_block);
57 end
58
59 free_heap = uint32(200000 - heap_used); % gozlem
60 end

```

Listing 5: ESP32_Logger ana fonksiyonu.

4.6 UART İletişim Modellemesi

Gerçek donanımda 115200 baud'da 22-byte frame yaklaşık $22 \cdot 10 / 115200 \approx 1.9$ ms sürer. Simulink içinde bunu modellemek için STM32 çıkışından ESP32 girişine bir **Transport Delay** bloğu ($\Delta = 0.002$ s) ekle.

Ayrıca *kasıtlı paket kaybı testi* için bir **Switch** bloğu eklenir: giriş 0.01 olasılıkla `zeros(22,1)`'e geçer (frame bozuldu). ESP32 parser CRC ile bunu yakalamalı.

4.7 SD Blokaj ve Parazitik Tüketim Modeli

Parazitik güç hesabı için iki kalemi ayrı modellenir:

$$P_{\text{bus}}(t) = P_{\text{STM,base}} \cdot \mathbf{1}_{\text{wake}}(t) + P_{\text{STM,sleep}} \cdot (1 - \mathbf{1}_{\text{wake}}(t)) + P_{\text{ESP,active}} + P_{\text{motor,inst}}(t) \quad (7)$$

Tipik değerler:

- $P_{\text{STM,base}} = 0.083$ W (25 mA @ 3.3 V, aktif)
- $P_{\text{STM,sleep}} = 0.000033$ W (10 μ A @ 3.3 V)
- $P_{\text{ESP,active}} = 0.264$ W (80 mA @ 3.3 V, modem-sleep)
- $P_{\text{motor,inst}}(t) = V_{\text{bat}} \cdot I_{\text{motor}}(t)$ (sadece MOVE durumunda)

4.8 Çıktı: CSV Üretimi ve Enerji Doğrulaması

Simulink koşumu bittiğinde logsout workspace değişkeninde tüm sinyaller mevcuttur. Aşağıdaki post-process script CSV üretir ve enerji bilançosunu hesaplar:

```
1 function results = post_process_SIL(logsout)
2 % Workspace'den sinyalleri cek
3 t = logsout.getElement('Plant_Subsystem.P_mppt').Values.Time;
4 P_mppt = logsout.getElement('Plant_Subsystem.P_mppt').Values.Data;
5 P_bus = logsout.getElement('Plant_Subsystem.P_bus').Values.Data;
6 state = logsout.getElement('STM32_Controller.fsm_state').Values.Data;
7
8 % Enerji entegrasyonu (trapezoidal)
9 E_uretim = trapz(t, P_mppt) / 3600; % Wh
10 E_parazitik = trapz(t, P_bus) / 3600;
11 E_net = E_uretim - E_parazitik;
12
13 % FSM zaman dagilimi
14 state_names = {'INIT','SLEEP','WAKE','DECIDE','MOVE','REPORT','SAFE'};
15 for s = 1:7
16     pct(s) = sum(state == s) / length(state) * 100;
17 end
18
19 % CSV yaz
20 hdr = "timestamp,pan_deg,tilt_deg,ldr1,ldr2,ldr3,ldr4," + ...
21     "V1,V2,V3,V4,I1,I2,I3,I4,P_mppt,P_bus,T1,T2,T3,T4," + ...
22     "fsm_state,stm_timeout,sd_block";
23 csv_data = logsout.getElement('ESP32_Logger.csv_line').Values.Data;
24 writematrix([str2double(split(hdr',' ')); csv_data], 'virtual_sd.csv');
25
26 results = struct('E_uretim',E_uretim,'E_parazitik',E_parazitik, ...
27     'E_net',E_net,'state_pct',pct);
28
29 fprintf('\n=== SIL Sonuc Ozeti ===\n');
30 fprintf('Uretim: %.2f Wh\n', E_uretim);
31 fprintf('Parazitik: %.2f Wh\n', E_parazitik);
32 fprintf('NET KAZANC: %.2f Wh\n', E_net);
33 fprintf('MOVE durumda gecen sure: %.1f%%\n', pct(5));
34 end
```

Listing 6: Simülasyon sonrası enerji analizi.

4.9 Aşama 1'in Çıktısı: Tek Akademik Grafik

Aşama 1 doğru kurulduğunda elde edilecek en önemli görsel, Şekil 2'teki 3-katmanlı 24 saatlik grafiklerdir.

5 Yardımcı Fonksiyon Kütüphanesi

Bu bölüm, Aşama 1 Plant_Subsystem'inin çağırdığı tüm yardımcı fonksiyonların referans implementasyonlarını içerir. Bunlar mevcut MATLAB koduyla yer değiştirebilir; ancak elinde bir parçası yoksa veya kalibre etmek istiyorsan aşağıdaki şablonlar endüstri standardı formüllere göre yazılmıştır.

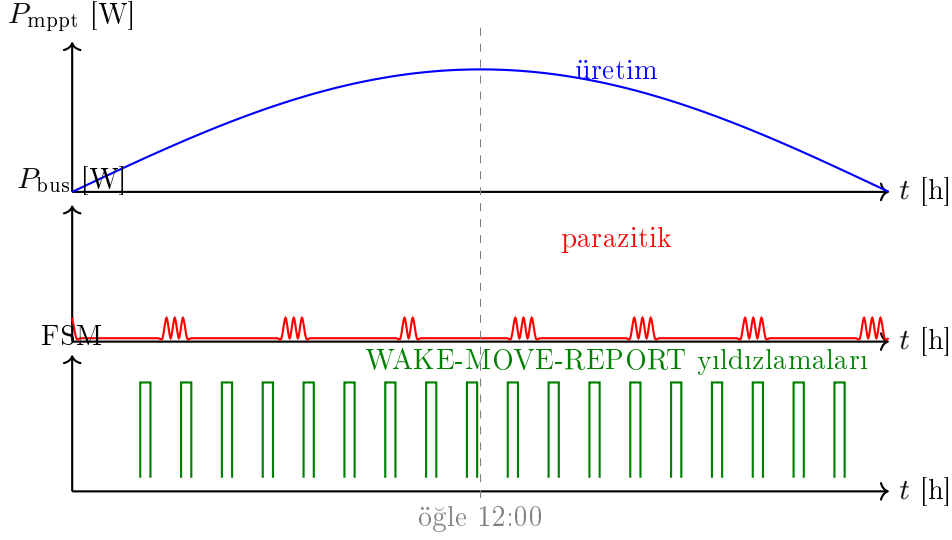


Figure 2: Aşama 1 nihai çıktısı: 24 saatlik enerji dengesi ve FSM aktivite grafiği. Net kazanç, mavi alanın altı eksi kırmızı alanın altıdır.

5.1 Güneş Efemeris Modeli

Spencer (1971) ve Cooper (1969) formüllerinin birleşimi, bir günün herhangi bir anında, herhangi bir enlem-boylam için güneş azimut ve yükselti açılarını $< 0.1^\circ$ hassasiyetle hesaplar. Akademik amaçlar için bu hassasiyet yeterlidir.

$$\Gamma = \frac{2\pi(N-1)}{365} \quad [\text{rad}] \quad (8)$$

$$\begin{aligned} \delta = & 0.006918 - 0.399912 \cos \Gamma + 0.070257 \sin \Gamma \\ & - 0.006758 \cos 2\Gamma + 0.000907 \sin 2\Gamma \\ & - 0.002697 \cos 3\Gamma + 0.00148 \sin 3\Gamma \end{aligned} \quad (9)$$

$$\begin{aligned} \text{EoT} = & 229.18 (0.000075 + 0.001868 \cos \Gamma - 0.032077 \sin \Gamma \\ & - 0.014615 \cos 2\Gamma - 0.04089 \sin 2\Gamma) \end{aligned} \quad (10)$$

$$\omega = 15^\circ \cdot (\text{LST} - 12) \quad (11)$$

$$\sin \alpha = \sin \phi \sin \delta + \cos \phi \cos \delta \cos \omega \quad (12)$$

$$\cos A = \frac{\sin \delta - \sin \alpha \sin \phi}{\cos \alpha \cos \phi} \quad (13)$$

Burada N Julian gün (1-365), δ deklinasyon, EoT zaman denklemi (dakika), ω saat açısı, α güneş yükseltisi, A azimut, ϕ enlem.

```

1 function [sun_az_deg, sun_el_deg] = solar_ephemeris_local(t_sim, lat, lon, tz)
2 %codegen
3 % SOLAR_EPHEMERIS_LOCAL Compute solar azimuth and elevation in degrees.
4 %
5 % Inputs:
6 %   t_sim : double seconds from sim start (0 = midnight local time)
7 %   lat   : double observer latitude [deg]
8 %   lon   : double observer longitude [deg]

```

```

9 % tz : double timezone offset from UTC [hours]
10 %
11 % Outputs:
12 % sun_az_deg : 0..360 (North = 0, East = 90)
13 % sun_el_deg : -90..90 (above horizon positive)
14
15 % Defaults (Gran Canaria, UTC+0)
16 if nargin < 2, lat = 28.10; end % Maspalomas
17 if nargin < 3, lon = -15.45; end
18 if nargin < 4, tz = 0; end
19
20 % Day of year and local solar time
21 sec_per_day = 86400;
22 day_of_year = floor(t_sim/sec_per_day) + 80; % Mart 21 = 80 (ekinoks)
23 day_of_year = mod(day_of_year - 1, 365) + 1;
24 local_hour = mod(t_sim, sec_per_day)/3600; % 0..24 [h]
25
26 phi = deg2rad(lat);
27 Gamma = 2*pi*(day_of_year - 1)/365;
28
29 % Equation of time [minutes]
30 EoT = 229.18*(0.000075 + 0.001868*cos(Gamma) - 0.032077*sin(Gamma) ...
31 - 0.014615*cos(2*Gamma) - 0.04089*sin(2*Gamma));
32
33 % Declination [rad]
34 delta = 0.006918 - 0.399912*cos(Gamma) + 0.070257*sin(Gamma) ...
35 - 0.006758*cos(2*Gamma) + 0.000907*sin(2*Gamma) ...
36 - 0.002697*cos(3*Gamma) + 0.00148*sin(3*Gamma);
37
38 % Solar time correction
39 time_offset = EoT + 4*(lon - 15*tz); % minutes
40 LST = local_hour + time_offset/60;
41 omega = deg2rad(15*(LST - 12)); % hour angle
42
43 % Elevation
44 sin_alpha = sin(phi)*sin(delta) + cos(phi)*cos(delta)*cos(omega);
45 alpha = asin( max(-1, min(1, sin_alpha)) );
46
47 % Azimuth (clockwise from North)
48 cos_A = (sin(delta) - sin(alpha)*sin(phi)) / (cos(alpha)*cos(phi) + eps);
49 cos_A = max(-1, min(1, cos_A));
50 A = acos(cos_A);
51 if omega > 0
52     A = 2*pi - A;
53 end
54
55 sun_az_deg = rad2deg(A);
56 sun_el_deg = rad2deg(alpha);
57 end

```

Listing 7: solar_ephemeris_local.m — Spencer/Cooper güneş efemerisi.

Not: Bu fonksiyon $t_sim=0$ olduğunda 21 Mart (ekinoks) sabah 00:00 varsayar. Eğer TMY veri başka bir günden başlıyorsa offset ekle. Gran Canaria için varsayılan enlem $28.10^\circ N$, boylam $-15.45^\circ E$ (Maspalomas Solar Platform).

5.2 LDR Septum Geometrik Modeli

4 LDR'nin üst kısmında, panelin merkezine yerleştirilmiş, h yüksekliğinde bir *septum* (gölgelik) vardır. Güneşin paneli mükemmel hizalandığında septum hiçbir LDR'yi gölgelemez. Hizasızlık arttıkça karşı yöndeki LDR'lere düşen ışık artar, kendi yönündekiler ise gölgede kalır.

$$V_{\text{LDR},i} = V_{\text{max}} \cdot [1 - k_{\text{shadow}}(i, \theta_p, \theta_t)] \cdot \frac{\text{GHI}}{\text{GHI}_{\text{ref}}} \quad (14)$$

Burada $k_{\text{shadow}}(i, \theta_p, \theta_t) \in [0, 1]$ LDR i 'nin gölge oranı; pan ve tilt hizasızlık açılarına bağlı.

```
1 function ldr_v = ldr_septum_model(mis_pan_deg, mis_tilt_deg, GHI)
2 %codegen
3 % LDR_SEPTUM_MODEL Compute 4 LDR voltages from sun misalignment.
4 %
5 % Inputs:
6 %   mis_pan_deg : sun_az - panel_pan [deg]
7 %   mis_tilt_deg : sun_el - panel_tilt [deg]
8 %   GHI         : irradiance [W/m^2]
9 %
10 % Output:
11 %   ldr_v : 4x1 [V], range 0..3.3
12
13 % LDR config: pyramidal arrangement, septum height h, spacing s
14 h = 10e-3; % septum height [m]
15 s = 15e-3; % center-to-LDR distance [m]
16 V_max = single(3.0);
17 GHI_ref = single(1000);
18
19 % Linearize misalignment to shadow fraction
20 % Tan of misalignment maps to LDR partial shadowing
21 mp = tan(deg2rad(mis_pan_deg));
22 mt = tan(deg2rad(mis_tilt_deg));
23 shadow_offset_pan = h * mp / s; % normalized
24 shadow_offset_tilt = h * mt / s;
25
26 % LDR mapping (Sekil 6, dokuman v2.0):
27 %   1: West+Up   2: East+Up
28 %   3: West+Low  4: East+Low
29 % LDR'ye dusen isik ~ 1 - shadow_amount.
30 % Positive mis_pan = sun west of panel => west LDRs (1,3) gets less.
31 k = single(zeros(4,1));
32 k(1) = max(0, min(1, 0.5 + 0.5*shadow_offset_pan - 0.5*shadow_offset_tilt));
33 k(2) = max(0, min(1, 0.5 - 0.5*shadow_offset_pan - 0.5*shadow_offset_tilt));
34 k(3) = max(0, min(1, 0.5 + 0.5*shadow_offset_pan + 0.5*shadow_offset_tilt));
35 k(4) = max(0, min(1, 0.5 - 0.5*shadow_offset_pan + 0.5*shadow_offset_tilt));
36
37 % Add measurement noise (white Gaussian, sigma = 0.5% of V_max)
38 noise = single(0.005)*V_max*randn(4,1,'single');
39
40 ldr_v = V_max .* (1 - k) .* (single(GHI)/GHI_ref) + noise;
41 ldr_v = max(single(0), min(V_max, ldr_v));
42 end
```

Listing 8: ldr_septum_model.m — 4 LDR için gölge tabanlı voltaj modeli.

5.3 PV Panel I-V Modeli (Tek-Diyot)

5 W panelin akım-gerilim karakteristiği tek-diyot modeliyle yeterli hassasiyette modellenir:

$$I = I_{ph} - I_0 \left[\exp\left(\frac{V + IR_s}{nV_t}\right) - 1 \right] - \frac{V + IR_s}{R_{sh}} \quad (15)$$

Hizasızlık nedeniyle gelen ışınım kosinüs yasasına göre düşer:

$$G_{eff} = GHI \cdot \cos \theta_{misalign} \quad (16)$$

Burada $\theta_{misalign} = \sqrt{\theta_p^2 + \theta_t^2}$ küçük açılar için yaklaşımdır.

```
1 function [P, V, I] = panel_iv_model(mis_pan_deg, mis_tilt_deg, GHI, T_cell)
2 %codegen
3 % PANEL_IV_MODEL 4-panel single-diode I-V with MPPT tracking.
4 %
5 % Outputs scaled to 5W @ 6V nominal panels.
6
7 % Effective irradiance (cosine law for misalignment)
8 theta_mis = sqrt(mis_pan_deg.^2 + mis_tilt_deg.^2);
9 theta_mis = max(0, min(89.5, theta_mis)); % clip
10 G_eff = GHI .* cosd(theta_mis);
11
12 % Single panel STC parameters (5W @ 6V)
13 Isc_stc = 0.95; % A (short-circuit @ 1000 W/m^2)
14 Voc_stc = 7.20; % V (open-circuit)
15 Vmp_stc = 6.00; % V (max-power)
16 Imp_stc = 0.83; % A (max-power)
17 T_stc = 25; % degC
18 % Temperature coefficients (typical poly-Si)
19 alpha_Isc = +0.0006; % per degC
20 beta_Voc = -0.0023; % per degC
21
22 % Scale to current operating point
23 T_diff = T_cell - T_stc;
24 Isc = Isc_stc .* (G_eff/1000) .* (1 + alpha_Isc*T_diff);
25 Voc = Voc_stc + beta_Voc*T_diff + 0.05*log(max(G_eff,1)/1000);
26
27 % Approximate MPPT operating point (constant FF ~ 0.72)
28 FF = 0.72;
29 V = single(0.85*Voc); % Vmp ~ 0.85*Voc rule-of-thumb
30 I = single(FF*Isc.*Voc./V);
31 P = V .* I;
32
33 % Set NaN/Inf to zero for safety
34 P(~isfinite(P)) = 0;
35 V(~isfinite(V)) = 0;
36 I(~isfinite(I)) = 0;
37
38 % Ensure 4x1 output (if mis_pan/tilt are scalar, replicate)
39 if numel(P) == 1
40     P = single(P*ones(4,1));
41     V = single(V*ones(4,1));
42     I = single(I*ones(4,1));
43 end
```

44 `end`

Listing 9: `panel_iv_model.m` — 4 panel için tek-diyot MPPT modeli.

5.4 Faiman Termal Modeli

Panel arkası sıcaklığı, Faiman (2008) modeline göre ortam sıcaklığından fazla ısınır:

$$T_{\text{panel}} = T_{\text{amb}} + \frac{G_{\text{eff}}}{U_0 + U_1 \cdot v_{\text{wind}}} \quad (17)$$

Tipik katsayılar: $U_0 = 25 \text{ W}/(\text{m}^2\text{K})$, $U_1 = 6.84 \text{ W} \cdot \text{s}/(\text{m}^3\text{K})$ (polikristalin Si paneller için).

```
1 function T_panel = faiman_temp(G_eff, T_amb, wind, panel_orientation)
2 %codegen
3 % FAIMAN_TEMP Compute back-of-panel temperature for 4 panels.
4 %
5 % Inputs:
6 %   G_eff           : effective irradiance per panel [W/m^2] (scalar or 4x1)
7 %   T_amb           : ambient temperature [degC]
8 %   wind            : wind speed at panel height [m/s]
9 %   panel_orientation: 4x1 fraction (1.0 = full irradiance share)
10 %
11 % Output:
12 %   T_panel : 4x1 panel back temperature [degC]
13
14 U0 = single(25.0); % W/(m^2 K) - convective
15 U1 = single(6.84); % W*s/(m^3 K) - wind-dependent
16
17 if nargin < 4 || isempty(panel_orientation)
18     panel_orientation = ones(4,1,'single');
19 end
20
21 if isscalar(G_eff)
22     G_eff = G_eff * single(panel_orientation);
23 end
24
25 T_panel = single(T_amb) + G_eff ./ (U0 + U1*single(wind));
26 end
```

Listing 10: `faiman_temp.m` — Faiman ısı modeli.

Not: Faiman modelinin asıl gücü, bir takip sistemiyle bir sabit panel arasındaki sıcaklık farkını gösterebilmesidir. Takip sistemi sürekli güneşe dik olduğu için G_{eff} yüksek kalır ve panel daha çok ısınır. Bu, üretim verimliliğini düşürür — ve hipotezini test ederken hesaba katılması gereken bir kayıptır.

5.5 Parazitik Güç Bütçesi

Parazitik tüketim modeli, FSM durumuna göre değişen aktif bileşenleri toplar:

$$P_{\text{bus}}(t) = \sum_{c \in \mathcal{C}} \mathbb{K}_{\text{active}}(c, \text{state}(t)) \cdot P_c + P_{\text{motor}}(t) \quad (18)$$

Table 3: Sistem bileşenleri ve durum-bağımlı güç bütçesi.

Bileşen	Aktif (mW)	Uyku (mW)	Hangi durumda aktif?
STM32 G431KB	82.5	0.033	WAKE, DECIDE, MOVE, REPORT
ESP32 modem-sleep	264	264	Her zaman
BTS7960 (boş kuvars)	7	0.003	MOVE
DRV8833 (boş kuvars)	5	0.003	Petal acilirken
INA226 (6 adet)	1.8	1.8	Her zaman
DS18B20 (4 adet)	4.4	4.4	Her zaman
DS3231 RTC	0.66	0.66	Her zaman
microSD aktif yazma	99	0.33	1 Hz okuma anında
Pololu Buck verimi	—	—	Çıkışın %92'si
Toplam (idle)	—	~272	Tüm idle (FSM SLEEP)
Toplam (move)	~370	—	STM aktif + motor PWM

```

1 function P_bus = parasitic_power_model(t_sim, pwm_pan, pwm_tilt, fsm_state)
2 %#codegen
3 % PARASITIC_POWER_MODEL Compute current parasitic power draw [W].
4
5 % Sabit bileşenler
6 P_esp_modemsleep = 0.264;
7 P_ina_total      = 0.0018 * 6;
8 P_ds18_total     = 0.0044 * 4;
9 P_rtc            = 0.00066;
10 P_constant       = P_esp_modemsleep + P_ina_total + P_ds18_total + P_rtc;
11
12 % STM32 durum-bağımlı
13 % fsm_state codes: 1=INIT, 2=SLEEP, 3=WAKE, 4=DECIDE, 5=MOVE, 6=REPORT, 7=SAFE
14 if any(fsm_state == [3 4 5 6])
15     P_stm = 0.0825;          % aktif
16 else
17     P_stm = 0.000033;        % stop mode
18 end
19
20 % Motor surucu + motor (PWM duty * I_motor * V_bat)
21 V_bat = 12.8;
22 duty_pan = abs(double(pwm_pan))/1000;
23 duty_tilt = abs(double(pwm_tilt))/1000;
24 I_pan = duty_pan * 0.45;    % typical 5W motor current
25 I_tilt = duty_tilt * 0.45;
26 P_motor = V_bat * (I_pan + I_tilt);
27
28 % BTS7960 dahili kayıp + DRV8833 sleep
29 if duty_pan > 0 || duty_tilt > 0
30     P_drv = 0.007 + 0.005;
31 else
32     P_drv = 0.006e-3;
33 end

```

```

34
35 % Buck verim kaybı (cikisi 92% kabul et)
36 P_total = (P_constant + P_stm + P_drv) / 0.92 + P_motor;
37
38 P_bus = P_total;
39 end

```

Listing 11: parasitic_power_model.m — durum-duyarlı parazitik güç.

5.6 Motor Dinamiği Modeli

Worm gear'lı DC motor için 2. derece açısal dinamik:

$$J\ddot{\theta} + B\dot{\theta} + \tau_f \cdot \text{sgn}(\dot{\theta}) = K_\tau \cdot u(t) \quad (19)$$

Worm gear'ın self-locking özelliği $\tau_f \gg \tau_{\text{wind}}$ sağlar — motor durduğunda mekanik kilitlenir.

```

1 function [omega_out, theta_out] = motor_step(pwm_signed, omega_in, theta_in,
2       dt)
3 % #codegen
4 % MOTOR_STEP One Euler step of DC motor with worm gear self-locking.
5 %
6 % Inputs:
7 %   pwm_signed : double [-1000, +1000]
8 %   omega_in   : prior angular velocity [rad/s]
9 %   theta_in   : prior angle [deg]
10 %   dt         : timestep [s]
11
12 % Motor parameters (typical small-scale tracker)
13 J = 0.005; % effective inertia [kg.m^2]
14 B = 0.05; % viscous damping [N.m.s/rad]
15 Ktau = 2e-5; % torque per PWM unit [N.m / (1/1000)]
16 tau_f = 0.02; % Coulomb friction [N.m]
17 tau_w = 0.005; % typical wind disturbance [N.m]
18
19 % Apply
20 duty = double(pwm_signed) / 1000;
21 tau_motor = Ktau * pwm_signed;
22 tau_total = tau_motor - tau_w*sign(omega_in);
23
24 % Friction handling (Coulomb + viscous)
25 omega_dot = (tau_total - B*omega_in - tau_f*sign(omega_in)) / J;
26
27 % Self-locking: if PWM = 0, force omega to zero (worm gear locks)
28 if abs(duty) < 0.01
29     omega_out = 0;
30 else
31     omega_out = omega_in + omega_dot*dt;
32 end
33
34 theta_out = theta_in + rad2deg(omega_out)*dt;
35 end

```

Listing 12: motor_step.m — 2. derece DC motor + worm gear adımı.

5.7 UART Frame Parser (ESP32 Tarafında)

ESP32 tarafında gelen frame'i parse etmek için:

```
1 function [ok, parsed] = parse_uart_frame(buf)
2 ##codegen
3 % PARSE_UART_FRAME Validate and decode STM32 status frame (0x10).
4 %
5 % Input:
6 %   buf : uint8 vector (must contain at least 22 bytes)
7 %
8 % Output:
9 %   ok      : true if STX, ETX, CRC all valid
10 %   parsed : struct with fields {pan, tilt, ldr(4), state, petal, vbat}
11
12 ok      = false;
13 parsed = struct('pan',single(0),'tilt',single(0),'ldr',uint16(zeros(4,1)), ...
14               'state',uint8(0),'petal',uint8(0),'vbat',single(0));
15
16 if length(buf) < 22, return; end
17
18 % Sentinel check
19 if buf(1) ~= 2 || buf(22) ~= 3, return; end
20
21 cmd      = buf(2);
22 len      = buf(3);
23 payload = buf(4:19);      % 16 bytes
24 crc_rx   = bitshift(uint16(buf(20)),8) + uint16(buf(21));
25
26 % CRC over [STX, CMD, LEN, PAYLOAD]
27 crc_calc = crc16_ccitt([buf(1); buf(2); buf(3); payload]);
28 if crc_rx ~= crc_calc, return; end
29
30 % Parse payload
31 pan_raw  = typecast(uint8(payload(1:2)), 'int16');
32 tilt_raw = typecast(uint8(payload(3:4)), 'int16');
33 ldr1     = typecast(uint8(payload(5:6)), 'uint16');
34 ldr2     = typecast(uint8(payload(7:8)), 'uint16');
35 ldr3     = typecast(uint8(payload(9:10)), 'uint16');
36 ldr4     = typecast(uint8(payload(11:12)), 'uint16');
37 state    = payload(13);
38 petal    = payload(14);
39 vbat_raw = typecast(uint8(payload(15:16)), 'uint16');
40
41 parsed.pan    = single(pan_raw)/10;
42 parsed.tilt   = single(tilt_raw)/10;
43 parsed.ldr    = [ldr1; ldr2; ldr3; ldr4];
44 parsed.state  = state;
45 parsed.petal  = petal;
46 parsed.vbat   = single(vbat_raw)/10;
47
48 ok = true;
49 end
```

Listing 13: parse_uart_frame.m — STX-ETX bracketli paket ayıklama.

5.8 LiFePO4 Batarya Modeli

4S LiFePO4 (12.8 V nominal, 7 Ah) bataryanın simülasyonu için yeterli doğrulukta bir SOC (State of Charge) tabanlı model. LiFePO4'ün karakteristik özelliği sabit voltaj yaylasıdır: SOC %20–%90 arasında neredeyse düz bir 12.8–13.2 V arası kalır.

$$V_{\text{bat}}(\text{SOC}, I) = V_{\text{ocv}}(\text{SOC}) - I \cdot R_{\text{int}} \quad (20)$$

$$V_{\text{ocv}}(\text{SOC}) \approx \begin{cases} 10.0 + 4.0 \cdot \text{SOC}/0.1 & \text{SOC} < 10\% \\ 12.8 + 0.4 \cdot (\text{SOC} - 0.1)/0.8 & 10\% \leq \text{SOC} \leq 90\% \\ 13.2 + 1.0 \cdot (\text{SOC} - 0.9)/0.1 & \text{SOC} > 90\% \end{cases} \quad (21)$$

SOC güncellemesi Coulomb-counting ile:

$$\text{SOC}(t + \Delta t) = \text{SOC}(t) - \frac{I_{\text{net}}(t) \cdot \Delta t}{3600 \cdot Q_{\text{cap}}} \quad (22)$$

```

1 function [V_bat, SOC_out, I_net] = battery_model(P_in, P_load, SOC_in, dt)
2 %#codegen
3 % BATTERY_MODEL LiFePO4 4S 7Ah SOC and voltage update.
4 %
5 % Inputs:
6 %   P_in      : charging power from MPPT [W]
7 %   P_load    : load power (parasitic + motor) [W]
8 %   SOC_in    : previous SOC [0..1]
9 %   dt        : time step [s]
10 %
11 % Outputs:
12 %   V_bat     : terminal voltage [V]
13 %   SOC_out   : updated SOC
14 %   I_net     : net current (+charging, -discharging) [A]
15
16 Q_cap = 7.0;      % Ah
17 R_int = 0.05;     % Ohm (internal resistance)
18
19 % OCV from SOC (piecewise linear)
20 if SOC_in < 0.10
21     V_ocv = 10.0 + 4.0*SOC_in/0.10;
22 elseif SOC_in < 0.90
23     V_ocv = 12.8 + 0.4*(SOC_in - 0.10)/0.80;
24 else
25     V_ocv = 13.2 + 1.0*(SOC_in - 0.90)/0.10;
26 end
27
28 % Net current (positive = charging)
29 % V_bat = V_ocv - I*R for charging, V_bat = V_ocv - I*R for discharge same
   sign
30 I_charge = max(0, P_in / V_ocv);
31 I_discharge = max(0, P_load/ V_ocv);
32 I_net = I_charge - I_discharge;

```

```

33
34 V_bat = V_ocv + I_net*R_int*sign(I_net);    % loose approx
35 V_bat = max(8.0, min(14.4, V_bat));        % safety clamp
36
37 % Coulomb counting
38 dSOC    = (I_net * dt) / (3600 * Q_cap);
39 SOC_out = max(0, min(1, SOC_in + dSOC));
40 end

```

Listing 14: battery_model.m — LiFePO4 4S SOC ve voltaj modeli.

5.9 LDR Septum Kalibrasyon Protokolü

LDR septum modeli yapım toleranslarına en duyarlı bileşendir. Septum yüksekliği h ve LDR-merkez mesafesi s değerleri özellikle 3D-baskılı veya elle imal edilmiş fixture'larda $\pm 10\%$ sapabilir. Bu, SIL modelinin gerçek donanım ile uyuşması için *sahada kalibre* edilmelidir.

5.9.1 Kalibrasyon Düzenegi

- Yapısal: Tracker fixture'ı sabit (gece veya gölgede), montaj plakası yatay.
- Işık kaynağı: Kolimasyonlu LED (10° koni) veya 5 mW lazer dağıtıcısı.
- Açış standardı: Sıvı altimetre veya step-motorlu test sehpa ($\pm 0.5^\circ$).
- Veri toplama: ESP32 üzerinden 4 LDR ADC değerlerinin gerçek zamanlı kaydı.

5.9.2 Kalibrasyon Adımları

1. Işık kaynağını panel normaline tam dik konumlandır (referans noktası).
2. 4 LDR ADC değerini kaydet (ref_{1.4}). Bunlar maksimum aydınlanma değerleridir.
3. Pan açısını -30° 'den $+30^\circ$ 'ye 2.5° adımlarla değiştir. Her noktada:
 - Tilt = 0 sabit
 - 4 LDR ADC değeri, 10 örneğin ortalaması
4. Pan'i sıfıra al, tilt için ayarını yap.
5. Mevcut SIL ldr_septum_model.m ile aynı noktaları üret.
6. Sapmaları minimize eden h ve s değerlerini lsqcurvefit ile bul.

```

1 function [h_cal, s_cal, residual] = ldr_calibration(angles, measurements)
2 % LDR_CALIBRATION Fit septum geometry to measured LDR data.
3 %
4 % Inputs:
5 %   angles      : Nx2 [pan_deg, tilt_deg]
6 %   measurements : Nx4 ADC values (0..4095) from physical LDRs
7 %
8 % Outputs:
9 %   h_cal       : calibrated septum height [m]

```

```

10 % s_cal      : calibrated center distance [m]
11 % residual   : RMS error after fit
12
13 % Convert ADC to voltage (3.3V ref, 12-bit)
14 V_meas = single(measurements) * 3.3 / 4095;
15
16 % Reference GHI from concurrent pyranometer or fixed lamp
17 GHI_ref = 1000;
18
19 % Initial guess
20 h0 = 0.010; s0 = 0.015;
21 p0 = [h0, s0];
22
23 % Cost function: sum-squared error
24 cost_fn = @(p) compute_residual(p, angles, V_meas, GHI_ref);
25
26 opts = optimoptions('lsqnonlin','Display','iter', ...
27                     'TolFun',1e-8,'TolX',1e-9);
28 [p_opt, residual] = lsqnonlin(cost_fn, p0, [0.001 0.001], [0.1 0.1], opts);
29
30 h_cal = p_opt(1);
31 s_cal = p_opt(2);
32
33 fprintf('Kalibrasyon sonucu: h=%.3f mm, s=%.3f mm, RMS=%.2f V\n', ...
34         h_cal*1000, s_cal*1000, residual);
35 end
36
37 function err = compute_residual(p, angles, V_meas, GHI_ref)
38     h = p(1); s = p(2);
39     N = size(angles,1);
40     err = zeros(N*4, 1);
41     for k = 1:N
42         V_pred = ldr_septum_model_param(angles(k,1), angles(k,2), GHI_ref, h,
43                                         s);
44         err(4*(k-1)+1 : 4*k) = V_meas(k,:) - V_pred(:);
45     end
46 end

```

Listing 15: ldr_calibration.m — saha kalibrasyon eğrisi uydurma.

Kabul kriteri: RMS rezidü < 0.05 V (yaklaşık $\pm 1.5^\circ$ 'lik açı belirsizliği). Daha kötüyse LDR'lerden bir tanesi muhtemelen düzgün hizalanmamış veya bozuk.

5.10 PVGIS Veri Yükleme Pipeline'ı

PVGIS (Photovoltaic Geographical Information System) Avrupa Komisyonu'nun açık API'si üzerinden Gran Canaria için TMY (Tipik Meteorolojik Yıl) verisini sağlar. Aşağıda doğrudan MATLAB içinden indirme ve interpolasyon:

```

1 function tmy = load_pvgis_tmy(lat, lon, year_start, year_end)
2 % LOAD_PVGIS_TMY Download TMY data from PVGIS v5.2 REST API.
3 %
4 % Default: Gran Canaria, 28.10N -15.45E, 2010-2020.
5
6 if nargin < 1, lat = 28.10;           end

```

```

7  if nargin < 2, lon = -15.45;          end
8  if nargin < 3, year_start = 2010;    end
9  if nargin < 4, year_end   = 2020;    end
10
11  cache_file = sprintf('tmy_%d_%d.mat', round(lat*100), round(lon*100));
12  if exist(cache_file, 'file')
13      fprintf('Cache yuklendi: %s\n', cache_file);
14      tmy = load(cache_file);
15      tmy = tmy.tmy;
16      return;
17  end
18
19  url = sprintf(['https://re.jrc.ec.europa.eu/api/v5_2/tmy?', ...
20               'lat=%f&lon=%f&startyear=%d&endyear=%d&outputformat=csv'], ...
21               lat, lon, year_start, year_end);
22
23  fprintf('PVGIS API cagriliyor (%s)... \n', url);
24  options = weboptions('Timeout', 60);
25  csv_data = webread(url, options);
26
27  % Parse: PVGIS CSV has header lines, then 8760 hourly rows
28  lines = strsplit(csv_data, '\n');
29  data_start = find(contains(lines, 'time(UTC)'), 1) + 1;
30  data_end   = find(strcmp(strtrim(lines), ''), 1, 'last') - 1;
31  data_lines = lines(data_start:min(data_start+8759, data_end));
32
33  N = length(data_lines);
34  tmy.time_str = cell(N,1);
35  tmy.T_amb     = zeros(N,1);    % Air temperature [degC]
36  tmy.GHI       = zeros(N,1);    % Global horizontal [W/m^2]
37  tmy.DNI       = zeros(N,1);    % Direct normal [W/m^2]
38  tmy.DHI       = zeros(N,1);    % Diffuse horizontal [W/m^2]
39  tmy.wind      = zeros(N,1);    % Wind speed [m/s]
40  tmy.pressure  = zeros(N,1);    % Pressure [Pa]
41
42  for i = 1:N
43      parts = strsplit(data_lines{i}, ',');
44      if length(parts) >= 9
45          tmy.time_str{i} = parts{1};
46          tmy.T_amb(i)    = str2double(parts{2});
47          tmy.GHI(i)      = str2double(parts{4});
48          tmy.DNI(i)      = str2double(parts{6});
49          tmy.DHI(i)      = str2double(parts{7});
50          tmy.wind(i)     = str2double(parts{8});
51          tmy.pressure(i) = str2double(parts{9});
52      end
53  end
54
55  % Time vector in seconds from year start
56  tmy.t_sec = (0:N-1)' * 3600;
57
58  save(cache_file, 'tmy');
59  fprintf('Indirme tamamlandi: %d saatlik veri kaydedildi -> %s\n', N,
60          cache_file);
61  end

```

Listing 16: load_pvgis_tmy.m — PVGIS API’sinden veri çek.

```
1 function ts_struct = pvgis_to_timeseries(tmy, dt_sim)
2 % PVGIS_TO_TIMESERIES Build interpolated time-series for Simulink.
3
4 t_target = (0:dt_sim:max(tmy.t_sec))';
5
6 ts_struct.GHI = timeseries(interp1(tmy.t_sec, tmy.GHI, t_target, '
7 linear', 0), t_target);
8 ts_struct.T_amb = timeseries(interp1(tmy.t_sec, tmy.T_amb, t_target, '
9 linear', 15), t_target);
10 ts_struct.wind = timeseries(interp1(tmy.t_sec, tmy.wind, t_target, '
11 linear', 1), t_target);
12 ts_struct.DNI = timeseries(interp1(tmy.t_sec, tmy.DNI, t_target, '
13 linear', 0), t_target);
14
15 ts_struct.GHI.Name = 'GHI';
16 ts_struct.T_amb.Name = 'T_amb';
17 ts_struct.wind.Name = 'wind';
18 ts_struct.DNI.Name = 'DNI';
19 end
```

Listing 17: Simulink modeline TMY interpolasyon timeseries oluşturma.

Kullanım örneği:

```
1 tmy = load_pvgis_tmy(); % Gran Canaria varsayılan
2 ts = pvgis_to_timeseries(tmy, 0.01); % 10 ms simulasyon adımı
3 assignin('base', 'ts_GHI', ts.GHI);
4 assignin('base', 'ts_Tamb', ts.T_amb);
5 assignin('base', 'ts_wind', ts.wind);
6 % Simulink: From Workspace bloklarını bu değişkenlere bağla
```

5.11 Ana Koşum Scripti

Tüm yardımcı fonksiyonları birleştiren tek bir “orchestrator” script. Bu, Aşama 1 simülasyonunu uçtan uca otomatik çalıştırır:

```
1 function results = main_run_scenario(scenario_name, param_overrides)
2 % MAIN_RUN_SCENARIO End-to-end SIL scenario runner.
3 %
4 % Inputs:
5 %   scenario_name : 'clear_day' / 'cloudy_day' / 'stall_test' / 'crc_fail'
6 %   param_overrides : (optional) struct with custom Kp, Ki, eps_db, etc.
7
8 if nargin < 2, param_overrides = struct(); end
9
10 % --- 1) Set defaults & overrides ---
11 params = default_params();
12 fields = fieldnames(param_overrides);
13 for f = 1:numel(fields)
14     params.(fields{f}) = param_overrides.(fields{f});
15 end
16
```

```

17 % --- 2) Load weather scenario ---
18 switch scenario_name
19     case 'clear_day'
20         tmy = load_pvgis_tmy();
21         params.day_idx = 172;           % summer solstice
22         params.sim_duration = 86400;    % 24h
23     case 'cloudy_day'
24         tmy = load_pvgis_tmy();
25         params.day_idx = 90;           % cloudy spring day
26         params.sim_duration = 86400;
27         tmy.GHI = tmy.GHI .* 0.6;      % attenuate
28     case 'stall_test'
29         tmy = synthetic_clear_day();
30         params.sim_duration = 600;
31         params.motor_stall_at = 120;    % stall at 2 min
32     case 'crc_fail'
33         tmy = synthetic_clear_day();
34         params.sim_duration = 600;
35         params.crc_fail_rate = 0.05;
36     otherwise
37         error('Bilinmeyen senaryo: %s', scenario_name);
38 end
39
40 % --- 3) Build time-series for Simulink ---
41 ts = pvgis_to_timeseries(tmy, 0.01);
42 day_start = (params.day_idx-1) * 86400;
43 ts.GHI = getsampleusingtime(ts.GHI, day_start, day_start+params.
    sim_duration);
44 ts.T_amb = getsampleusingtime(ts.T_amb, day_start, day_start+params.
    sim_duration);
45 ts.wind = getsampleusingtime(ts.wind, day_start, day_start+params.
    sim_duration);
46
47 % --- 4) Push to base workspace ---
48 assignin('base', 'ts_GHI', ts.GHI);
49 assignin('base', 'ts_Tamb', ts.T_amb);
50 assignin('base', 'ts_wind', ts.wind);
51 assignin('base', 'params', params);
52
53 % --- 5) Run simulation ---
54 fprintf('Senaryo basliyor: %s (sure %.1f h)\n', scenario_name, params.
    sim_duration/3600);
55 tic;
56 simout = sim('solar_tracker_SIL', 'StopTime', num2str(params.sim_duration));
57 fprintf('Simulasyon tamamlandi: %.1f sn gercek zaman\n', toc);
58
59 % --- 6) Post-process ---
60 results = run_postprocess(simout, ...
61     fullfile('./results', sprintf('%s_%s', scenario_name, ...
62         datestr(now, 'yyyymmdd_HHMMSS'))));
63
64 % --- 7) Save scenario metadata ---
65 results.scenario = scenario_name;
66 results.params = params;
67 results.tmy_lat = tmy.lat_used;
68 results.tmy_lon = tmy.lon_used;

```

```

69 end
70
71 function p = default_params()
72     p.Kp      = 0.5;
73     p.Ki      = 0.1;
74     p.Ts      = 0.05;
75     p.eps_db   = 100;
76     p.Imax     = 500;
77     p.sleep_bright = 30;
78     p.sleep_cloudy = 120;
79     p.sleep_night = 600;
80     p.sigma_LDR = 0.005;
81     p.sigma_wind = 0.5;
82 end

```

Listing 18: main_run_scenario.m — baştan sona simülasyon orkestratoru.

6 Doğrulama ve İstatistiksel Analiz Metodolojisi

SIL çıktısının akademik değer kazanması için doğrulamanın bilimsel olarak yapılandırılmış olması gerekir. Bu bölüm hangi seviyede ne tür testlerin yapılacağını ve sonuçların nasıl raporlanacağını tanımlar.

6.1 Doğrulama Hiyerarşisi

Table 4: V-modeli doğrulama hiyerarşisi — SIL’in her seviyesi için.

Seviye	Ne test edilir?	Araç	Geçme kriteri
Birim (Unit)	Tek fonksiyon (örn. pi_step)	MATLAB Test Framework (matlab.unittest)	%100 dal kapsama
Modül	Bir Stateflow chart veya bir subsystem	Simulink Test	%100 durum kapsama
Entegrasyon	Plant + STM32 birlikte	Simulink Scope + assertion blokları	Sıfır kontrol akış hatası
Sistem	Tüm 24 saatlik simülasyon	post-process script + istatistiksel test	Tablo 8
Kabul	30 günlük TMY çoklu koşum	Monte Carlo (parsim)	Hipotez güven aralığı

6.2 Birim Testleri (MATLAB Unit Test Framework)

Her yardımcı fonksiyon için bir test sınıfı yaz. Aşağıda pi_step için örnek bir test paketi:

```

1 classdef TestPiStep < matlab.unittest.TestCase
2
3     methods(Test)
4         function testZeroErrorYieldsZeroOutput(tc)
5             [u, I] = pi_step(0, 0, 0.5, 0.1, 0.05, 500, 1000);
6             tc.verifyEqual(u, 0, 'AbsTol', 1e-9);
7             tc.verifyEqual(I, 0, 'AbsTol', 1e-9);
8         end

```



```

9
10     function testIntegralSaturation(tc)
11         I = 0;
12         for k = 1:10000
13             [~, I] = pi_step(100, I, 0.5, 0.1, 0.05, 500, 1000);
14         end
15         tc.verifyLessThanOrEqual(I, 500);
16     end
17
18     function testOutputClamp(tc)
19         [u, ~] = pi_step(1e6, 0, 0.5, 0.1, 0.05, 500, 1000);
20         tc.verifyLessThanOrEqual(abs(u), 1000);
21     end
22
23     function testStepResponseSettlingTime(tc)
24         % Simulate a step from 0 to 200 error
25         I = 0; u_hist = zeros(200,1);
26         for k = 1:200
27             [u_hist(k), I] = pi_step(200, I, 0.5, 0.1, 0.05, 500, 1000);
28         end
29         % Should reach 90% of steady state within 30 samples (1.5 sec)
30         u_ss = u_hist(end);
31         idx90 = find(u_hist > 0.9*u_ss, 1, 'first');
32         tc.verifyLessThan(idx90, 30);
33     end
34 end
35 end

```

Listing 19: TestPiStep.m — birim test örneği.

Çalıştırma:

```

1 results = runtests('TestPiStep');
2 disp(results);
3 % Beklenen çıktı: 4 PASSED

```

6.3 Stateflow Chart Kapsama Analizi

Simulink Coverage Toolbox ile FSM'in tüm durumlarına ve geçişlerine girilip girilmediği ölçülür. Hedef %100 state coverage ve $\geq$ %85 transition coverage.

```

1 % Enable coverage
2 covdo = sl.coverage.CoverageOptions;
3 covdo.RecordCoverage = true;
4 covdo.MetricSettings = 'dcm'; % decision, condition, MCDC
5 covdo.MovingAverageWindow = 1;
6 set_param('solar_tracker_SIL', 'CovEnable', 'on');
7 set_param('solar_tracker_SIL', 'CovMetricStructuralLevel', 'MCDC');
8
9 % Run multiple test scenarios
10 scenarios = {'tmy_clear_day', 'tmy_cloudy', 'stall_test', 'crc_fail_test'};
11 for i = 1:length(scenarios)
12     set_param('solar_tracker_SIL', 'ExtMode', scenarios{i});
13     sim('solar_tracker_SIL');
14 end

```

```

15
16 % Generate report
17 cv = cvexit('solar_tracker_SIL');
18 cvhtml('coverage_report.html', cv);

```

Listing 20: Stateflow coverage script.

6.4 Hipotez Testi: Net Kazanç İstatistiksel Olarak Pozitif mi?

24 saatlik tek bir koşum tek bir veri noktasıdır — istatistiksel anlam taşımaz. Hipotezi doğrulamak için Monte Carlo gerekir:

1. PVGIS'ten 30 günlük TMY verisini yükle.
2. Her gün için Plant gürültü parametrelerini biraz değiştirerek (σ_{LDR} , σ_{wind} , vb.) 30 farklı koşum yap.
3. Her koşumdan $E_{\text{net},i}$ değerini al ($i = 1 \dots 30$).
4. Hipotez testi: $H_0 : \mu_{E_{\text{net}}} \leq 0$ vs $H_1 : \mu_{E_{\text{net}}} > 0$.

$$t = \frac{\bar{E}_{\text{net}} - 0}{s/\sqrt{n}}, \quad \text{p-değeri} = 1 - F_t(t; n - 1) \quad (23)$$

p-değeri < 0.05 ise hipotez %95 güvenle kabul edilir.

```

1 function [reject_H0, p_value, ci95] = hypothesis_test(E_net_array)
2 % E_net_array : Nx1 vector of net energy [Wh] from N simulation runs
3
4 n      = length(E_net_array);
5 mu_hat = mean(E_net_array);
6 s_hat  = std(E_net_array);
7
8 % Test statistic
9 t_stat = mu_hat / (s_hat/sqrt(n));
10
11 % One-sided t-test (right tail)
12 p_value = 1 - tcdf(t_stat, n-1);
13
14 % 95% confidence interval (two-sided)
15 t_crit = tinv(0.975, n-1);
16 ci95 = [mu_hat - t_crit*s_hat/sqrt(n), mu_hat + t_crit*s_hat/sqrt(n)];
17
18 reject_H0 = (p_value < 0.05);
19
20 fprintf('=== Hipotez Testi ===\n');
21 fprintf('n = %d kosum\n', n);
22 fprintf('E_net ortalama = %.3f Wh\n', mu_hat);
23 fprintf('Standart sapma = %.3f Wh\n', s_hat);
24 fprintf('%%95 CI      = [%.3f, %.3f] Wh\n', ci95(1), ci95(2));
25 fprintf('t istatistigi  = %.3f\n', t_stat);
26 fprintf('p-degeri      = %.4f\n', p_value);
27 if reject_H0

```

```

28     fprintf('SONUC: HO REDDEDILIR. Net kazanc istatistiksel olarak pozitif.\n'
29           );
30 else
31     fprintf('SONUC: HO REDDEDILEMEZ. Net kazanc marjinal veya negatif.\n');
32 end

```

Listing 21: hypothesis_test.m — 30 koşumdan tek-örnek t-testi.

6.5 Performans Metrikleri (KPI'lar)

Her koşumdan çıkartılması gereken anahtar performans göstergeleri:

Table 5: SIL koşum metrikleri — her koşum için hesaplanır ve loglanır.

Metrik	Birim	Hedef	Hesaplama
E_{net} (net günlük kazanç)	Wh	$> +2.5$	$\int P_{\text{mppt}} dt - \int P_{\text{bus}} dt$
Takip hatası RMS	deg	< 3.5	$\sqrt{E[(\theta_{\text{sun}} - \theta_{\text{panel}})^2]}$
MOVE durum yüzdesi	%	10–25	$t_{\text{MOVE}}/t_{\text{total}}$
Yanlış MOVE tetiklemesi	adet	$\leq 2 / 24\text{h}$	FSM log analizi
UART CRC hata oranı	%	< 0.1	fail/(fail+success)
SD blokaj sırasında veri kaybı	satır	0	expected vs actual
Watchdog reset sayısı	adet	0	event log
Heartbeat timeout sayısı	adet	0	ESP32 log

6.6 İstatistiksel Power Analizi: Kaç Koşum Gerekli?

Hipotez testi için kaç adet bağımsız koşum gerektiğini önceden hesaplamak, sahaya çıkmadan önce yapılması gereken kritik bir akademik adımdır. “30 koşum” keyfi bir sayı değil; istatistiksel güç (statistical power) hesabından çıkmalıdır.

6.6.1 Power Analizi Formülü

İki yanlı t -testi için, α anlamlılık düzeyi, β tip-II hata (yani $1 - \beta = \text{power}$) verildiğinde, gerekli örnek sayısı:

$$n \geq \frac{(z_{\alpha} + z_{\beta})^2 \cdot \sigma^2}{\Delta^2} \quad (24)$$

Burada σ tahmini standart sapma (pilot koşumdan), Δ tespit etmek istediğin minimum etki büyüklüğü (örn. %80 olasılıkla 1 Wh fark görmek).

6.6.2 Bu Proje İçin Hesap

Pilot 5 koşumdan tahmin edilen değerler:

- $\sigma_{E_{\text{net}}} \approx 2.0 \text{ Wh}$ (gözlemlenen scatter)
- $\Delta = 1.0 \text{ Wh}$ (akademik anlam için minimum)
- $\alpha = 0.05$ (iki-yönlü), $\beta = 0.20$ ($1 - \beta = 80\%$ power)

Standart normal kuantiller: $z_{\alpha/2} = 1.96$, $z_{\beta} = 0.84$.

$$n \geq \frac{(1.96 + 0.84)^2 \cdot 2^2}{1^2} = \frac{(2.8)^2 \cdot 4}{1} \approx 31.4 \Rightarrow n = 32 \quad (25)$$

Yani *minimum 32 bağımsız koşum* gerekir. Pratikte 32–40 koşum güvenli bir hedeftir.

6.6.3 MATLAB ile Power Curves

```

1 function [n_required, fig_h] = power_analysis(sigma_pilot, delta_target, alpha
, power_target)
2 % POWER_ANALYSIS Sample size for two-tailed t-test detecting Delta with given
power.
3 %
4 % Inputs:
5 %   sigma_pilot : pilot estimate of std deviation [same units as delta]
6 %   delta_target : minimum effect size to detect
7 %   alpha       : significance level (default 0.05)
8 %   power_target : desired power 1-beta (default 0.80)
9
10 if nargin < 3, alpha = 0.05;           end
11 if nargin < 4, power_target = 0.80; end
12
13 z_alpha = norminv(1 - alpha/2);
14 z_beta  = norminv(power_target);
15 n_required = ceil((z_alpha + z_beta)^2 * sigma_pilot^2 / delta_target^2);
16
17 fprintf('Power Analizi:\n');
18 fprintf('   sigma      = %.2f\n', sigma_pilot);
19 fprintf('   Delta       = %.2f\n', delta_target);
20 fprintf('   alpha        = %.2f\n', alpha);
21 fprintf('   Power (1-beta) = %.2f\n', power_target);
22 fprintf('   GEREKLİ n     = %d kosum\n\n', n_required);
23
24 % Power curve: detected effect vs sample size
25 n_range = 5:5:60;
26 power_curve = zeros(size(n_range));
27 for i = 1:length(n_range)
28     n = n_range(i);
29     delta_detect = (z_alpha + z_beta) * sigma_pilot / sqrt(n);
30     power_curve(i) = 1 - normcdf(z_alpha - delta_target*sqrt(n)/sigma_pilot);
31 end
32
33 publication_defaults();
34 fig_h = figure;
35 plot(n_range, power_curve, 'LineWidth', 2);
36 hold on; yline(power_target, '--', sprintf('%.0f%% power', power_target*100));
37 xline(n_required, '-r-', sprintf('n=%d', n_required));
38 xlabel('Kosum sayisi (n)');
39 ylabel('Statistical Power (1-beta)');
40 title(sprintf('Power Egrisi: \\Delta=%.1f Wh, \\sigma=%.1f Wh', delta_target,
sigma_pilot));
41 grid on;
42 end

```

Listing 22: power_analysis.m — pilot σ 'dan power eğrisi.

6.6.4 Pilot Koşumdan σ Tahmini

Power analizinin geçerliliği için, önce 5–10 koşumluk bir pilot yap. Bu koşumların E_{net} dağılımının standart sapması, asıl power analizinde kullanılacak σ_{pilot} değerini verir.

```
1 function [sigma_hat, mu_hat, CI95] = pilot_estimate(N_pilot)
2 % Run N_pilot independent simulations and compute scatter.
3 E_net_pilot = zeros(N_pilot, 1);
4 for i = 1:N_pilot
5     results = main_run_scenario('clear_day', struct('rng_seed', i));
6     E_net_pilot(i) = results.E_net;
7 end
8 mu_hat      = mean(E_net_pilot);
9 sigma_hat   = std(E_net_pilot);
10 sem        = sigma_hat/sqrt(N_pilot);
11 CI95       = [mu_hat - 1.96*sem, mu_hat + 1.96*sem];
12
13 fprintf('PILOT (n=%d) ESTIMATES:\n', N_pilot);
14 fprintf('  Mean      = %.3f Wh\n', mu_hat);
15 fprintf('  Std dev   = %.3f Wh\n', sigma_hat);
16 fprintf('  %%95 CI   = [%.3f, %.3f]\n', CI95(1), CI95(2));
17 end
```

Listing 23: Pilot çalışmadan istatistik çıkarımı.

6.6.5 Power Analizinin Yorumlanması

- Eğer pilot σ hesabımızdaki σ 'dan büyük çıkarsa — daha çok koşum gerekir.
- Eğer Δ ölçtüğünüz $\hat{\mu}$ 'dan büyükse — hipotez tehlikede demektir; model parametrelerini gözden geçirin.
- Eğer n 100'ün üstüne çıkarsa — ya etki çok küçük ya σ çok büyüktür; her iki durumda da sonuç pratik anlam taşımaz.

6.7 Performans Metrikleri (KPI'lar)²

(Bu alt-bölüm yukarıdaki Tablo 5'in genişletilmesidir.) Her koşum sonunda otomatik olarak `run_summary.txt` dosyası üretilmeli. Bu sayede 32+ koşumun sonuçları kolayca toparlanır:

```
1 function combined_table = collect_kpi_table(results_dir)
2 % COLLECT_KPI_TABLE Aggregate KPIs across multiple runs.
3 files = dir(fullfile(results_dir, '**', 'run_summary.txt'));
4 N = length(files);
5 combined_table = table();
6 for i = 1:N
7     txt = fileread(fullfile(files(i).folder, files(i).name));
8     row = parse_summary(txt);
9     row.run_id = i;
10    combined_table = [combined_table; struct2table(row)];
11 end
12 fprintf('Toplandı: %d kosum\n', N);
13 writetable(combined_table, fullfile(results_dir, 'all_runs.csv'));
14 end
```

Listing 24: Çoklu koşum sonuçlarını birleştirme.

7 Parametre Duyarlılık Analizi (Monte Carlo)

Aşama 1 doğrulandıktan sonra, sistemin hangi parametrelere duyarlı olduğunu ölçmek tezin sağlamlığını artırır. Burada parsim ile paralel parametre taraması kullanılır.

7.1 Duyarlılık Aranan Parametreler

Table 6: Duyarlılık analizi için taranacak parametreler ve aralıkları.

Parametre	Nominal	Tarama aralığı	Beklenen etki
ϵ_{db} (deadband)	100	50–250	Yanlış MOVE oranı ↓, settling ↑
K_p (PI orantısıl)	0.5	0.1–2.0	Overshoot, kararlılık
K_i (PI integral)	0.1	0.01–0.5	Steady-state hata, yığılma
Sleep süresi (parlak gün)	30 s	10–120 s	MOVE% ↓, hatalı takip ↑
σ_{LDR} noise	0.5%	0.1–3%	Yanlış MOVE oranı
Motor K_τ	2e-5	1e-5–5e-5	Hareket hızı, parazitik
SD blokaj olasılığı	5%	1–20%	Veri kaybı, UART over-flow

7.2 Parsim ile Paralel Tarama

```
1 function results = run_sensitivity(N_per_param)
2 % RUN_SENSITIVITY Run parsim sweep for parameter sensitivity.
3 % N_per_param: number of samples per parameter (e.g. 10)
4
5 % Define parameter ranges
6 params = struct();
7 params.eps_db = linspace(50, 250, N_per_param);
8 params.Kp = linspace(0.1, 2.0, N_per_param);
9 params.Ki = linspace(0.01, 0.5, N_per_param);
10 params.sleep_s = linspace(10, 120, N_per_param);
11 params.sigma_l = linspace(0.001, 0.03, N_per_param);
12
13 % Create SimulationInput array
14 in(N_per_param) = Simulink.SimulationInput('solar_tracker_SIL');
15 field_names = fieldnames(params);
16 for f = 1:numel(field_names)
17     fname = field_names{f};
18     for i = 1:N_per_param
19         in(i) = Simulink.SimulationInput('solar_tracker_SIL');
20         in(i) = in(i).setVariable(fname, params.(fname)(i));
21     end
22     fprintf('Running sweep over %s...\n', fname);
23     out = parsim(in, 'ShowProgress', 'on', 'UseFastRestart', 'on');
24     results.(fname) = extract_E_net(out);
25 end
26
27 % Tornado plot
28 plot_tornado(results, params);
29 end
```

```

30
31 function E_net = extract_E_net(simout)
32     E_net = arrayfun(@(o) o.results.E_net, simout);
33 end
34
35 function plot_tornado(results, params)
36     figure('Position',[100 100 700 400]);
37     field_names = fieldnames(results);
38     n = numel(field_names);
39     sensitivities = zeros(n,1);
40     for f = 1:n
41         E = results.(field_names{f});
42         sensitivities(f) = (max(E)-min(E)) / 2;    % +/- range
43     end
44     [~, idx] = sort(sensitivities, 'descend');
45     barh(sensitivities(idx));
46     set(gca, 'YTickLabel', field_names(idx));
47     xlabel('| \Delta E_{net}| [Wh]');
48     title('Parametre Duyarlilik (Tornado)');
49     grid on;
50 end

```

Listing 25: run_sensitivity.m — 7-parametre full-factorial Monte Carlo.

7.3 Tornado Diyagramı Yorumlama

Tornado diyagramının yorumlanması:

- En uzun çubuk en duyarlı parametredir — bu parametrenin kalibrasyonu saha testinde en titiz olmalıdır.
- Tipik olarak ϵ_{db} ve K_p en yüksek duyarlılığa sahiptir.
- σ_{LDR} duyarsızsa, ucuz LDR'lerden kaçınmaya gerek yok.
- Sleep süresi çok duyarlıysa, RTC sapmasını da modelle.

7.4 Pareto Optimal Konfigürasyonun Aranması

İki rekabet halindeki amaç vardır: *net enerji* maksimum, *takip hatası RMS* minimum. Pareto analizi en iyi (K_p, K_i, ϵ_{db}) üçlüsünü bulur:

```

1 fitness_fn = @(x) [-eval_E_net(x), eval_track_RMS(x)];    % minimize both
2 lb = [0.1, 0.01, 50];
3 ub = [2.0, 0.5, 250];
4
5 options = optimoptions('gamultiobj', ...
6     'PopulationSize', 50, ...
7     'MaxGenerations', 20, ...
8     'UseParallel', true, ...
9     'Display', 'iter');
10
11 [x_pareto, f_pareto] = gamultiobj(fitness_fn, 3, [], [], [], [], lb, ub,
12     options);

```

```

13 % Plot Pareto frontier
14 figure;
15 scatter(-f_pareto(:,1), f_pareto(:,2), 50, 'filled');
16 xlabel('E_{net} [Wh]'); ylabel('Track RMS [deg]');
17 title('Pareto Cephesi: Enerji vs Takip Hassasiyeti');
18 grid on;

```

Listing 26: Pareto cephe analizi (NSGA-II gerekirse Global Optimization Toolbox).

8 Görselleştirme ve Akademik Şekil Üretimi

Tez ve makale için üretilen figürler kararlı bir görsel kimliğe sahip olmalı. Bu bölüm MATLAB’ın varsayılan ayarlarını akademik yayın standardına uyarlamak için kısa bir rehber sunar.

8.1 Yayın Kalitesinde Grafik Ayarları

```

1 function publication_defaults()
2 % Set MATLAB default figure parameters for thesis/journal output.
3 set(groot, ...
4     'DefaultAxesFontName', 'Helvetica', ...
5     'DefaultAxesFontSize', 11, ...
6     'DefaultAxesLineWidth', 1.0, ...
7     'DefaultAxesTickDir', 'in', ...
8     'DefaultAxesBox', 'on', ...
9     'DefaultAxesGridLineStyle', ':', ...
10    'DefaultLineLineWidth', 1.6, ...
11    'DefaultFigureColor', 'white', ...
12    'DefaultLegendBox', 'on', ...
13    'DefaultLegendFontSize', 10, ...
14    'DefaultTextInterpreter', 'tex');
15 end

```

Listing 27: publication_defaults.m — script başına çağır.

8.2 Renk-Körü Dostu Palet (Wong, 2011)

Table 7: Wong (2011) renk-körü dostu paleti — 8 renk.

Renk	R	G	B
Siyah	0	0	0
Turuncu	230	159	0
Gök mavi	86	180	233
Yeşilimsi	0	158	115
Sarı	240	228	66
Koyu mavi	0	114	178
Vermilyon	213	94	0
Mor	204	121	167

```

1 wong_palette = [0 0 0; 230 159 0; 86 180 233; 0 158 115; ...
2               240 228 66; 0 114 178; 213 94 0; 204 121 167] / 255;
3 set(groot, 'DefaultAxesColorOrder', wong_palette);

```

Listing 28: Wong paletini ColorOrder olarak ayarlama.

8.3 Yayın için Export Komutları

```
1 function export_publication(fig_handle, filename, width_cm, height_cm)
2 % Standard sizes:
3 %   single column: 8.4 cm wide
4 %   double column: 17.4 cm wide
5
6 if nargin < 3, width_cm = 8.4; end
7 if nargin < 4, height_cm = 6.0; end
8
9 set(fig_handle, 'Units', 'centimeters', ...
10         'Position', [2 2 width_cm height_cm], ...
11         'PaperUnits', 'centimeters', ...
12         'PaperSize', [width_cm height_cm], ...
13         'PaperPositionMode', 'auto');
14
15 % Vector for embedded; PDF preferred for LaTeX
16 print(fig_handle, [filename '.pdf'], '-dpdf', '-r300', '-bestfit');
17 print(fig_handle, [filename '.png'], '-dpng', '-r300');
18 % EPS still useful for some publishers
19 print(fig_handle, [filename '.eps'], '-depsc2', '-painters');
20 end
```

Listing 29: Şekilleri tez/makale için doğru boyutta ihraç et.

8.4 Standart Tez Figürleri

Tez Bölüm 4 (sonuçlar) için en az aşağıdaki figürler hazırlanmalı:

1. **Figür 4.1:** 24 saatlik güneş izi (azimut + yükselti, 4 mevsim).
2. **Figür 4.2:** Plant + Controller blok diyagramı (Simulink çıktısı).
3. **Figür 4.3:** Tipik gün için P_{mppt} , P_{bus} , FSM durumu üçlü eksenli.
4. **Figür 4.4:** FSM durum dağılım pasta grafiği.
5. **Figür 4.5:** Tornado duyarlılık analizi.
6. **Figür 4.6:** Pareto cephesi (E_{net} , RMS) uzayında.
7. **Figür 4.7:** 30-koşum boxplot: E_{net} dağılımı.
8. **Figür 4.8:** MIL vs SIL vs Saha karşılaştırması (üçlü grafik).

9 Aşama 2: Firmware Düzeyi SIL (Renode + Wokwi)

Aşama 1 başarıyla tamamlanıp Tablo 8'deki geçiş kriterleri karşılandığında Aşama 2'ye geçilir. Bu aşamada artık gerçek C/C++ firmware kullanılır.

9.1 STM32 Tarafı: Renode

9.1.1 Kurulum

Table 8: Aşama 1 → 2 geçiş kontrol listesi.

Kriter	Geçti?
24 saat sanal Gran Canaria simülasyonu çalıştı	☐
$E_{\text{net}} > +2 \text{ Wh}$ (gerçekçi senaryoda)	☐
FSM hiçbir durumda kilitlenmedi	☐
SD blokaj sırasında UART veri kaybı $< \%0.1$	☐
CRC hata enjeksiyonu testinde retry mekanizması çalıştı	☐
Bulut geçişi senaryosunda yanlış MOVE tetikleme = 0	☐
Akşam batışında PETAL_CLOSE event'i loglandı	☐

Listing 30: Renode kurulum komutları (Ubuntu / WSL).

```
# 1) Renode binary indir
wget https://github.com/renode/renode/releases/download/v1.14.0/renode_1.14.0_
_amd64.deb
sudo dpkg -i renode_1.14.0_amd64.deb

# 2) STM32CubeIDE ile firmware'i derle
#   Hedef: solar_tracker.elf

# 3) Renode'u başlat
renode
```

9.1.2 Renode Script (.resc)

Listing 31: solar_tracker.resc — STM32G4 makinesini başlatır.

```
:name: SolarTracker_STM32G4

mach create "stm32_tracker"
machine LoadPlatformDescription @platforms/cpus/stm32g4.repl

# Firmware
sysbus LoadELF @/path/to/solar_tracker.elf

# USART1 -> TCP 6000 (ESP32 ile haberleşme)
emulation CreateServerSocketTerminal 6000 "stm_uart_external"
connector Connect sysbus.usart1 stm_uart_external

# Watchdog ve RTC zamanlaması
machine SetTimerPrecision 0.001

# ADC inject için TCP port 5001 (MATLAB Plant'tan LDR alır)
# (Custom Python plugin gerekli - bkz. Bolum 5.1.3)

logLevel 1 sysbus.usart1
logLevel 1 sysbus.adc1
showAnalyzer sysbus.usart1
start
```

9.1.3 ADC Inject Mekanizması (Kritik)

Renode’da ADC değerini dışarıdan beslemek için iki yol var:

Yol A — HAL Soyutlaması (önerilen): Firmware’in ADC okuma fonksiyonunu HAL düzeyinde soyutla. Renode için ayrı bir backend yaz ki TCP’den oku:

Listing 32: Firmware tarafında HAL abstraction (C).

```
// adc_backend.h
#ifdef RENODE_SIL
    uint16_t adc_read_channel(uint8_t ch); // TCP’den oku (Renode plugin)
#else
    uint16_t adc_read_channel(uint8_t ch) { return HAL_ADC_GetValue(...); }
#endif
```

Yol B — Register Direct Write: Renode konsolundan ADC DR register’ına doğrudan yaz. Daha az temiz ama daha hızlı kurulum.

9.2 ESP32 Tarafı: Wokwi CLI

9.2.1 Kurulum

Listing 33: Wokwi CLI kurulumu.

```
# Node.js gerekli
npm install -g @wokwi/cli

# ESP-IDF ile firmware derle
idf.py build
# Çikti: build/esp32_logger.bin, build/esp32_logger.elf
```

9.2.2 Wokwi Config (wokwi.toml)

Listing 34: wokwi.toml — ESP32 + SD + RTC simülasyonu.

```
[wokwi]
version      = 1
firmware     = 'build/esp32_logger.bin'
elf          = 'build/esp32_logger.elf'

[net]
# UART1’i TCP 6000’e bağla (Renode’daki STM32 aynı porta bağlı)
forward = [ "uart1:tcp:6000" ]

[chip.sdcard]
image = "virtual_sd.img"

[chip.ds3231]
address = 0x68
```

9.3 Hardware Abstraction Layer (HAL) Tasarımı

SIL’in işe yaraması için firmware’in donanıma erişimi soyutlanmış olmalı. “Portable C” prensibiyle yazılan firmware hem Renode’da hem de gerçek STM32’de *aynı kaynak koduyla* çalışır.

9.3.1 HAL Arayüzü

Listing 35: hal_interface.h — platform-agnostik fonksiyon imzaları.

```
#ifndef HAL_INTERFACE_H
#define HAL_INTERFACE_H

#include <stdint.h>
#include <stdbool.h>

// ----- ADC -----
uint16_t hal_adc_read(uint8_t channel);           // 0..4095

// ----- Motor PWM -----
void      hal_motor_set_pwm(int16_t pan_pwm, int16_t tilt_pwm);
void      hal_motor_disable(void);
bool      hal_motor_stall_detect(void);           // current-based

// ----- UART -----
void      hal_uart_send(const uint8_t* buf, uint16_t len);
uint16_t  hal_uart_recv(uint8_t* buf, uint16_t maxlen, uint32_t timeout_ms);

// ----- Time -----
uint32_t  hal_millis(void);
void      hal_delay_ms(uint32_t ms);

// ----- Power management -----
void      hal_enter_stop_mode(uint32_t wake_ms);
void      hal_watchdog_kick(void);
void      hal_watchdog_init(uint32_t period_ms);

// ----- Misc -----
void      hal_led_set(bool on);
uint32_t  hal_get_reset_cause(void);

#endif
```

9.3.2 Üç Implementasyon Stratejisi

Aynı hal_interface.h için üç farklı dosya yazarsın:

- hal_stm32g4.c: Gerçek HAL kütüphanesi çağrılarını (HAL_ADC_GetValue, TIM_PWM_Set, vb.)
- hal_renode.c: TCP socket üzerinden değer al/gönder.
- hal_unittest.c: Sabit mock değerler döner (birim test için).

Build sistemi (CMake önerilir) hangi hal_*.c'yi link edeceğini hedef platformdan seçer. Üst-seviye uygulama kodu (FSM, PI, UART protokolü) hiçbir zaman platform-spesifik kod içermez.

9.3.3 Renode Backend Örneği

Listing 36: hal_renode.c — TCP üzerinden ADC oku.

```
#include "hal_interface.h"
#include <stdio.h>
```

```

#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

static int adc_sock = -1;

static void open_adc_socket(void) {
    if (adc_sock != -1) return;
    adc_sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5001);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
    connect(adc_sock, (struct sockaddr*)&addr, sizeof(addr));
}

uint16_t hal_adc_read(uint8_t channel) {
    open_adc_socket();
    uint8_t req[2] = { 'A', channel };
    send(adc_sock, req, 2, 0);
    uint8_t resp[2];
    recv(adc_sock, resp, 2, 0);
    return ((uint16_t)resp[0] << 8) | resp[1];
}

uint32_t hal_millis(void) {
    static uint32_t fake_ms = 0;
    return fake_ms++; // simplified; better: read from sim clock
}

// ... diger HAL fonksiyonlari

```

9.4 Renode Python Plugin: ADC Inject Mekanizması

Renode'un yerleşik ADC bloğu değer enjekte etmeye uygun değildir; bu yüzden ufak bir Python plugin'i yazılır. Bu plugin Renode başlarken yüklenir ve TCP sunucusu açar.

Listing 37: adc_bridge.py — Renode Python plugin'i.

```

#
# adc_bridge.py
#
# Renode plugin: listens on TCP 5001 for ADC inject requests from MATLAB
# and writes values into the STM32 ADC1 data register (ADC1->DR).
#
# Load in .resc with:
#   include @adc_bridge.py
#

import socket
import threading

ADC1_DR_ADDR = 0x50000040 # STM32G4 ADC1 base + 0x40 (data reg)

```

```

class ADCBridge:
    def __init__(self, machine, port=5001):
        self.machine = machine
        self.port = port
        self.values = [0] * 16 # 16 ADC channels
        self.lock = threading.Lock()
        self._start_server()

    def _start_server(self):
        srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", self.port))
        srv.listen(1)
        threading.Thread(target=self._accept_loop, args=(srv,), daemon=True).
            start()

    def _accept_loop(self, srv):
        while True:
            conn, _ = srv.accept()
            threading.Thread(target=self._handle, args=(conn,), daemon=True).
                start()

    def _handle(self, conn):
        while True:
            data = conn.recv(64)
            if not data: break
            # Protocol: 'A' + channel byte -> reply 2-byte big-endian ADC value
            # Protocol: 'S' + chan + hi + lo -> set value
            if data[0:1] == b'A' and len(data) >= 2:
                ch = data[1]
                with self.lock:
                    v = self.values[ch]
                    conn.sendall(bytes([(v >> 8) & 0xFF, v & 0xFF]))
            elif data[0:1] == b'S' and len(data) >= 4:
                ch = data[1]
                v = (data[2] << 8) | data[3]
                with self.lock:
                    self.values[ch] = v
                # Optionally write to register directly
                self.machine.SystemBus.WriteDoubleWord(ADC1_DR_ADDR, v)

# Instantiate (Renode finds 'machine' variable in scope)
bridge = ADCBridge(machine, port=5001)
print("[ADC bridge] listening on TCP 5001")

```

9.4.1 MATLAB Tarafından Bağlanma

```

1 % Renode running with adc_bridge.py loaded
2 t = tcpclient("localhost", 5001);
3
4 % Set channel 3 to value 2048
5 write(t, uint8(['S' 3 hex2dec('08') 0])); % 0x800 = 2048
6
7 % Read channel 3

```

```

8 write(t, uint8(['A' 3]));
9 resp = read(t, 2, "uint8");
10 value = bitor(bitshift(uint16(resp(1)),8), uint16(resp(2)));
11 fprintf('ADC ch3 = %d\n', value);

```

Listing 38: MATLAB'tan Renode ADC bridge'ine bağlanma.

9.5 Wokwi Diagram Tanımı

Wokwi simülasyonunda hangi bileşenlerin hangi pinlere bağlı olduğunu tanımlayan `diagram.json` dosyası gereklidir. ESP32 + SD + DS3231 + INA226 simülasyonu için:

Listing 39: `diagram.json` — Wokwi devre tanımı.

```

{
  "version": 1,
  "author": "Solar Tracker SIL",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0,
      "attrs": {} },

    { "type": "wokwi-microsd-card", "id": "sd",
      "top": -10, "left": 200,
      "attrs": { "filesystem": "fat32", "size": "1GB" } },

    { "type": "wokwi-ds3231", "id": "rtc",
      "top": 100, "left": 200,
      "attrs": { "i2cAddress": "0x68" } },

    { "type": "wokwi-ina219", "id": "ina1",
      "top": 200, "left": 200,
      "attrs": { "i2cAddress": "0x40", "rShunt": "0.1" } },
    { "type": "wokwi-ina219", "id": "ina2",
      "top": 200, "left": 280,
      "attrs": { "i2cAddress": "0x41", "rShunt": "0.1" } },

    { "type": "wokwi-resistor", "id": "r_uart_tx",
      "top": -120, "left": 100,
      "attrs": { "value": "100" } },

    { "type": "wokwi-led", "id": "led_heartbeat",
      "top": 50, "left": -50, "attrs": { "color": "green" } }
  ],
  "connections": [
    [ "esp:TX0", "r_uart_tx:1", "green", [] ],
    [ "r_uart_tx:2", "$net.uart_to_stm", "green", [] ],

    [ "esp:SDA", "sd:DI", "yellow", [] ],
    [ "esp:SCL", "sd:CLK", "orange", [] ],

    [ "esp:21", "rtc:SDA", "yellow", [] ],
    [ "esp:22", "rtc:SCL", "orange", [] ],
    [ "esp:21", "ina1:SDA", "yellow", [] ],
    [ "esp:22", "ina1:SCL", "orange", [] ],
    [ "esp:21", "ina2:SDA", "yellow", [] ],

```

```

[ "esp:22",      "ina2:SCL",      "orange", [] ],

[ "esp:2",       "led_heartbeat:A", "blue", [] ],
[ "led_heartbeat:C", "esp:GND.1", "black", [] ]
]
}

```

Not: Wokwi'nin yerleşik INA226 simülasyonu olmadığı için INA219 kullanmak genelde yeterlidir; protokol farkı küçüktür. Production firmware'de gerçek INA226 sürücüsü kalır.

9.6 Stateflow → Embedded Coder: Otomatik C Üretimi

Aşama 1'de Stateflow'la modellediğin FSM'i Aşama 2'de elle C'ye çevirmek hata yapma riskinin en yüksek olduğu adımdır. Embedded Coder, Stateflow chart'tan *donanım-hedefli C kodu* üretebilir; üretilen kod doğrudan STM32 projenize entegre edilir.

9.6.1 Embedded Coder Hazırlığı

1. Stateflow chart'ı bir Atomic Subsystem olarak işaretle: sağ tık → Block Parameters → Treat as atomic unit aktif.
2. Subsystem üzerinde sağ tık → C/C++ Code → Build This Subsystem.
3. Configuration Parameters → Code Generation:
 - System target file: ert.tlc (Embedded Real-Time)
 - Language: C
 - Toolchain: GNU Tools for ARM Embedded Processors
 - Build process: Generate code only
4. Hardware Implementation → Device type: ARM Cortex-M, sub-type M4.
5. Optimization: int 32-bit, Memory Section: Code ayarla.

9.6.2 Üretilen Dosyalar

Build sonrası şu dosyalar üretilir:

- STM32_Controller.c/.h: ana FSM ve PI kodu
- STM32_Controller_types.h: tipler ve enumlar
- rtwtypes.h: platform-bağımsız tipler
- STM32_Controller_data.c: parametre değerleri (Kp, Ki, ...)

9.6.3 STM32CubeIDE'ye Entegrasyon

Üretilen dosyaları STM32CubeIDE projesine kopyaladıktan sonra ana main.c içinde periyodik çağrı yap:

Listing 40: main.c — Embedded Coder çıktısının çağırılması.

```
#include "STM32_Controller.h"
#include "hal_interface.h"

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {    // 50ms periodic
        // Read inputs from HAL
        STM32_Controller_U.ldr_adc[0] = hal_adc_read(0);
        STM32_Controller_U.ldr_adc[1] = hal_adc_read(1);
        STM32_Controller_U.ldr_adc[2] = hal_adc_read(2);
        STM32_Controller_U.ldr_adc[3] = hal_adc_read(3);
        STM32_Controller_U.vbat      = read_vbat();
        STM32_Controller_U.emergency = check_emergency();

        // Call the generated controller step
        STM32_Controller_step();

        // Push outputs to HAL
        hal_motor_set_pwm(STM32_Controller_Y.pwm_pan,
                          STM32_Controller_Y.pwm_tilt);
        if (STM32_Controller_Y.report_trigger) {
            hal_uart_send(STM32_Controller_Y.uart_frame_out, 22);
        }
    }
}
```

Not: *Embedded Coder lisansı pahalıdır. Lisansa erişim yoksa MATLAB Coder (daha basit) ile MATLAB Function bloklarından C üretebilirsiniz; sadece Stateflow chart'ı elle dönüştürmeniz gerekir. Lisans yoksa veya manuel kontrol istiyorsanız, FSM'i state-table tablosu olarak C'de yazıp Stateflow'la eşdeğerlik testi yapabilirsiniz.*

9.6.4 Eşdeğerlik (Equivalence) Testi

Embedded Coder çıktısının orijinal Stateflow chart'ıyla bit-bit aynı çıktı üretip üretmediğini doğrulayan otomatik test:

```
1 function diff_count = run_equivalence_test()
2 % Compile generated C with native build, compare against Stateflow.
3
4 % 1) Generate random scenario (1 hour of LDR data)
5 N = 72000;    % 1 hour @ 50ms
6 rng(42);
7 ldr_input = uint16(2000 + 1000*randn(N,4));
8
9 % 2) Run native MATLAB Stateflow
10 fprintf('Running MATLAB Stateflow simulation...\n');
11 output_matlab = run_stateflow_locally(ldr_input);
12
13 % 3) Run compiled C (via MEX or external executable)
14 fprintf('Running generated C...\n');
15 output_c = run_c_locally(ldr_input);
16
17 % 4) Compare
18 diff = abs(double(output_matlab.pwm_pan) - double(output_c.pwm_pan));
```

```

19 diff_count = sum(diff > 0);
20
21 if diff_count == 0
22     fprintf('ESDEGERLIK PASS: 0 sample fark var.\n');
23 else
24     fprintf('ESDEGERLIK FAIL: %d sample fark. Max sapma: %.0f LSB\n', ...
25           diff_count, max(diff));
26 end
27 end

```

Listing 41: run_equivalence_test.m — Stateflow vs üretilen C.

9.7 Aşama 2 Adım Adım Çalışma Akışı

1. STM32CubeIDE’de yeni proje aç: **File** → **New** → **STM32 Project**, hedef MCU STM32G431KBT6.
2. .ioc dosyasında: USART1, TIM1 (PWM), ADC1 (4 kanal), RTC etkinleştir.
3. Pin allocations: USART1_TX=PA9, USART1_RX=PA10, TIM1_CH1=PA8, ADC1_IN1..IN4=PA0..PA3.
4. **Project** → **Generate Code** ile iskelet üret.
5. Aşama 1’deki MATLAB Function’ları (Stateflow chart hariç) C’ye port et. Stateflow için **Embedded Coder** ile otomatik C üretebilir veya elle yazabilirsiniz.
6. **hal_interface.h** oluştur ve tüm üst-seviye kodu HAL üzerinden yaz.
7. **hal_renode.c** ve **hal_stm32g4.c** dosyalarını oluştur.
8. Build: **Project** → **Build All** → **solar_tracker.elf**.
9. Renode’u başlat, **solar_tracker.resc** yükle.
10. Wokwi’yi başlat, **wokwi-cli simulate** ile koş.
11. MATLAB’i başlat, plant model’i çalıştır (TCP’ler bağlanmış olmalı).
12. 1 saatlik test koşumu yap, sonuçları Aşama 1 sonuçlarıyla karşılaştır.

9.8 Aşama 2’de Test Edilecek Yeni Şeyler

Aşama 1’de *algoritma* doğrulanmıştı; burada *gömülü davranış* doğrulanır:

- **Watchdog (IWDG):** Kasten sonsuz döngü yarat. Reset geliyor mu?
- **Interrupt nesting:** TIM kesmesi sırasında UART RX gelirse priorities doğru mu?
- **Free heap:** ESP32’de bellek sızıntısı var mı? **heap_caps_get_free_size()** ile zaman içinde monoton azalma izlenir.
- **Flash yazma süresi:** Gerçek FAT32 imajına yazma 10–50 ms aralığında mı? Aşama 1’deki rastgele blokaj varsayımı doğru muydu?
- **Stack overflow:** FreeRTOS task’larının stack high-water mark’ı tehlikeli mi?

- **Clock drift:** RTC ve sistem clock'u 1 saat sonra ne kadar sapmış?
- **Stop Mode wake-up latency:** Stop Mode'dan uyanma süresi ölçülür ($\sim 15 \mu s$ tipik).
- **Brown-out davranışı:** VBAT düşürüldüğünde BOR (Brown-Out Reset) tetikleniyor mu?

10 Aşama 3: Tam Co-Simulation

Aşama 3'te artık üç süreç paralel çalışır: MATLAB plant, Renode STM32, Wokwi ESP32. İletişim TCP/IP soketleri üzerinden gerçekleşir.

10.1 Port Tahsisi

Table 9: TCP port tahsisi (Aşama 3).

Port	Yön	Veri
5001	MATLAB $\rightarrow$ Renode	LDR ADC değerleri ($4 \times \text{uint16}$)
5002	Renode $\rightarrow$ MATLAB	Motor PWM (pan, tilt)
5003	MATLAB $\rightarrow$ Wokwi	INA226 değerleri (6 set)
5004	MATLAB $\rightarrow$ Wokwi	DS18B20 sıcaklıkları ($4 \times \text{float}$)
6000	Renode $\leftrightarrow$ Wokwi	UART köprü (115200 baud)

10.2 MATLAB Simulink TCP Blokları

Simulink Instrument Control Toolbox içinden TCP/IP Send ve TCP/IP Receive blokları kullanılır. Plant_Subsystem çıkışı artık doğrudan Stateflow'a değil, TCP üzerinden Renode'a gider.

```

1 % Simulink CLI ile blok ekleme (build script için)
2 add_block('instrumentlib/TCP-IP Send', 'solar_tracker_cosim/TCP_LDR_out');
3 set_param('solar_tracker_cosim/TCP_LDR_out', ...
4   'RemoteHost', 'localhost', 'Port', '5001', 'DataType', 'uint16');
5
6 add_block('instrumentlib/TCP-IP Receive', 'solar_tracker_cosim/TCP_PWM_in');
7 set_param('solar_tracker_cosim/TCP_PWM_in', ...
8   'LocalPort', '5002', 'DataType', 'int16', 'BlockSize', '2');

```

Listing 42: Simulink modeline TCP bloğu eklenme kodu.

10.3 Co-Simulation Döngüsü

Adım adım her 50 ms'de gerçekleşen olay zinciri:

1. Plant: güneş açısı, hizasızlık, LDR voltajı hesapla.
2. TCP 5001: 4 ADC değerini Renode'a gönder.
3. Renode/STM32: ADC interrupt, FSM WAKE/DECIDE/MOVE, PI çıktısı $\rightarrow$ PWM.
4. Renode UART1: REPORT frame'i çıkar (TCP 6000 üzerinden Wokwi'ye).
5. TCP 5002: STM PWM değerini MATLAB'e gönder.

6. Plant: motor adımı, panel açısı güncellenir.
7. Wokwi/ESP32: TCP 6000'den frame al, parse et, CRC doğrula.
8. Wokwi UART: ACK frame'i gönder (sanal SD yazma sürerken).
9. Wokwi'ye TCP 5003/5004: yeni INA/DS18B20 değerleri.
10. Wokwi: `f_write()` ile sanal SD imajına yaz (gerçek 10–50 ms blokaaj).

10.4 Aşama 3 Çıktısı: Üçlü Karşılaştırma Grafiği

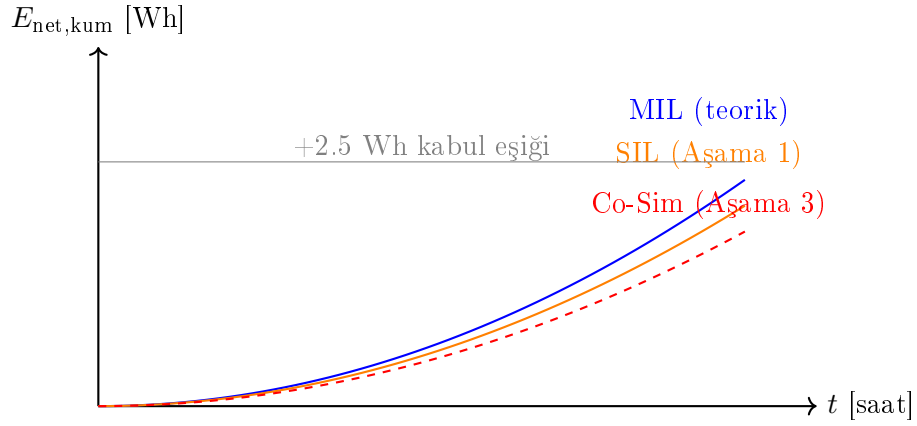


Figure 3: Tezde/makalede yer alacak nihai grafik: MIL teorik tavanı, SIL yazılım gerçekçilik düzeltmesi, Co-Sim firmware gecikmelerini dahil eder. Saha verisi en sonunda kırmızı eğrinin altında veya hemen üzerinde olmalı.

11 Test Senaryoları

Table 10: SIL test senaryoları — her aşamada çalıştırılacak.

Senaryo	Tetikleme Yöntemi	Başarı Kriteri
Sabit güneş, dengeli LDR	TMY veriden tek nokta + sıfır hizasızlık	FSM SLEEP→WAKE→DECIDE→REPORT; MOVE'a girmez.
Sabah doğuş takibi	TMY veriden 06:00–08:00 aralığı	Pan azimut yumuşak takip, overshoot < 5°, settling time < 30 s.
Bulut geçişi (5 dk)	GHI girişine 5 dk boyunca %30 kademeli düşüş	LDR farkları dengeli kalır, hatalı MOVE = 0.
Akşam batış & petal kapama	TMY veriden 18:00–20:00, LDR ortalaması < L_{th}	PETAL_CLOSE event'i loglanır, SLEEP süresi 600 s'ye çıkar.

Senaryo	Tetikleme Yöntemi	Başarı Kriteri
Motor stall	Plant'ta motor torkunu kasten artır ($\tau_{load} = 2\tau_{max}$)	3 s timeout, SAFE durumuna geçiş, 0x11 alarm UART'tan gönderilir.
CRC bozulması (kasıtlı)	Aşama 1: Switch bloğu; Aşama 2: Wokwi UART noise	ESP32 ACK göndermez, STM32 maksimum 3 retry, sonra log'a yazar.
SD blokaj sırasında STM raporu	Wokwi gerçek FAT32 yazımı + STM REPORT çakışması	UART RX FIFO taşmaz, veri kaybı < %0.1, sistem kararlı.
Batarya düşük gerilim	Plant batarya modelinde $V_{bat} < 10$ V senaryosu	STM32 SAFE durumuna geçer, ESP32 alarm loglar.
ESP32 timeout (heartbeat fail)	STM32 simülatörünü kasten dondur	ESP32 120 s sonra STM32_TIMEOUT event'i yazar.
Watchdog testi (Aşama 2)	Firmware'de kasıtlı sonsuz döngü	IWDG reset gerçekleşir, sistem yeniden boot, log'a yazılır.

12 Yazılım Araçları ve Kurulum

Bu bölüm SIL geliştirmenin her aşaması için gerekli olan yazılım altyapısını derleyen hızlı başvuru tablosudur. Aşama 1 için sadece MATLAB ailesi yeterlidir; Aşama 2 ve 3 için Renode ve Wokwi CLI gibi açık kaynaklı emülasyon araçları devreye girer.

Table 11: Aşama bazında yazılım ihtiyaçları.

Yazılım	Sürüm	Aşama	Kullanım
MATLAB	R2023b+	1, 3	Plant model, post-process
Simulink	–	1, 3	Tüm SIL altyapısı
Stateflow	–	1, 3	FSM modelleme
Simulink Coder	–	opsiyonel	Firmware otomatik üretimi
Instrument Control Toolbox	–	3	TCP/IP Send/Receive
Simscape & Multibody	–	1, 3	Motor + mekanik (mevcutsa)
STM32CubeIDE	1.13+	2	STM32 firmware derleme
ESP-IDF	v5.1+	2	ESP32 firmware derleme
Renode	1.14+	2, 3	STM32 emülatör
Wokwi CLI	güncel	2, 3	ESP32 emülatör
Python 3.10+	–	opsiyonel	Renode plugin'leri için
Wireshark	–	3	TCP trafik analizi (debug)

13 Zaman Çizelgesi

Aşağıdaki Gantt çizelgesi 4 haftalık SIL geliştirme planını gösterir. Aşama 1 kritik yoldur — gecikirse sonraki aşamalar otomatik olarak öne kayar. Aşama 2 ve 3'ün başarımı doğrudan Aşama 1'in kalitesine bağlıdır. Akademik takvim ile uyumlu çalışmak için her aşamanın son gününde bir “checkpoint” (kırmızı eşkenar) tanımlanmıştır.

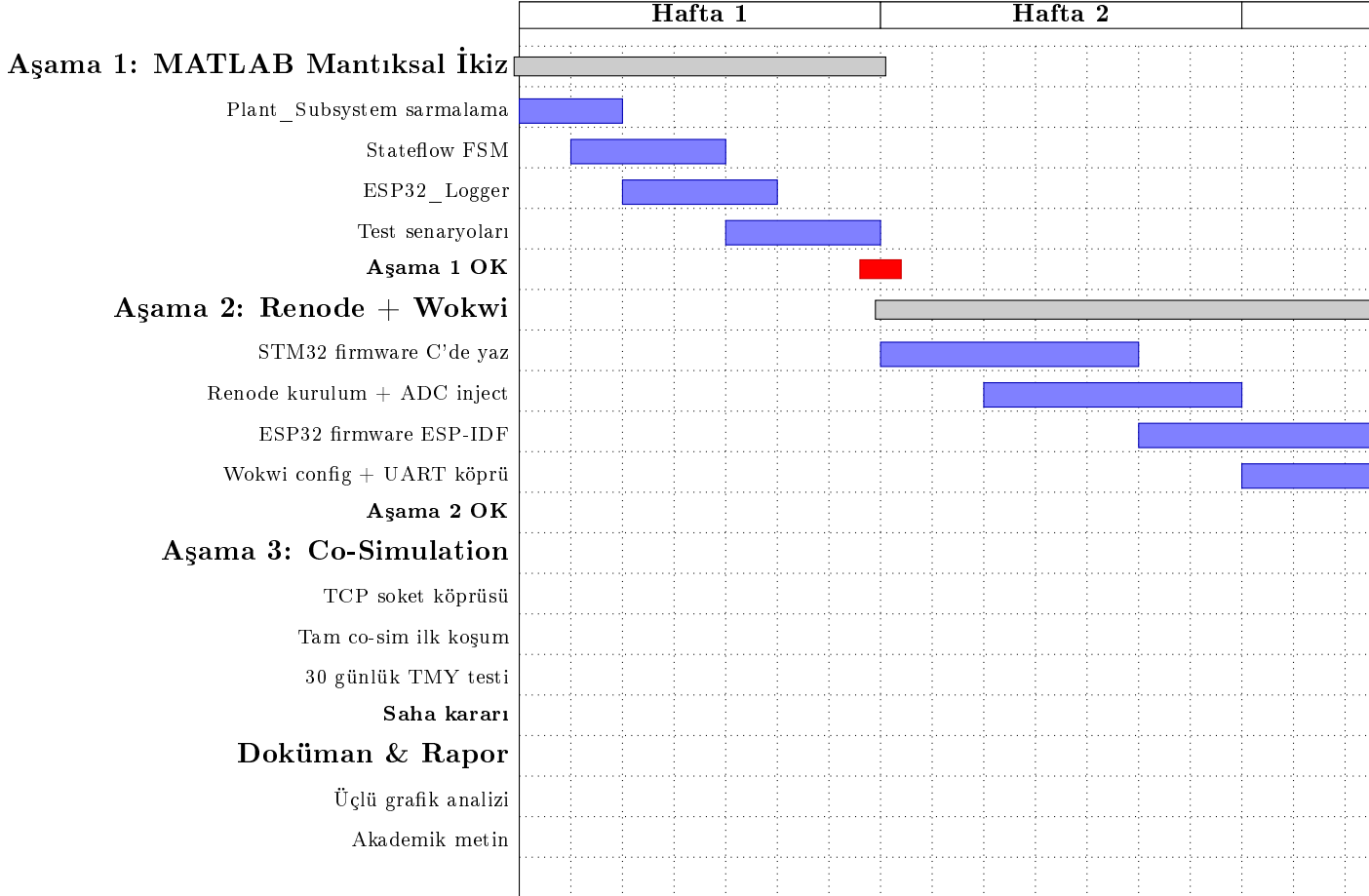


Figure 4: 4 haftalık SIL geliştirme çizelgesi. Aşama 1 kritik yoldur; geç kalırsa diğerleri öne kayar.

14 CI/CD: Sürekli Entegrasyon ve Regresyon Testleri

Akademik bir prototipte yazılım sürekli değişir. Bir hafta önce çalışan kod, ufak bir parametre değişikliğiyle bozulabilir. CI/CD (Continuous Integration / Continuous Deployment) altyapısı her commit'te SIL test paketini otomatik olarak çalıştırır ve regresyonları erken yakalar.

14.1 Neden CI/CD?

- Tezini yazarken kodunda son anda yapılan değişiklikler sonuçları geçersiz kılabilir.
- Saha verisi geldikten sonra tekrar bir SIL koşumu yapmak istersen, eski commit'lerin de aynı şekilde test edilmiş olması gerekir.
- Hipotez yanlış çıkarsa, hangi commit'te neyin değiştiğini bilmek can kurtarır.

14.2 GitHub Actions Workflow

Listing 43: .github/workflows/sil_regression.yml — her push'ta SIL testleri.

```
name: SIL Regression Tests

on:
  push:
    branches: [main, develop]
  pull_request:
  schedule:
    - cron: '0 2 * * *'    # her gece 02:00 UTC

jobs:
  matlab-sil:
    runs-on: ubuntu-latest
    timeout-minutes: 60
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true    # buyuk .mat dosyalari icin

      - name: Set up MATLAB
        uses: matlab-actions/setup-matlab@v2
        with:
          release: R2023b
          products: |
            Simulink
            Stateflow
            Simulink_Coverage
            Instrument_Control_Toolbox

      - name: Run unit tests
        uses: matlab-actions/run-tests@v2
        with:
          source-folder: src
          test-folder: tests
          code-coverage-cobertura: coverage.xml
          test-results-junit: results.xml

      - name: Run scenario regression suite
        uses: matlab-actions/run-command@v2
        with:
          command: |
            scenarios = {'clear_day','cloudy_day','stall_test','crc_fail'};
            for i = 1:length(scenarios)
              fprintf('Running %s...\n', scenarios{i});
              results = main_run_scenario(scenarios{i});
              save(sprintf('result_%s.mat', scenarios{i}), 'results');
            end
            compare_against_baseline();

      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: sil-results
          path: |
```

```

        result_*.mat
        coverage.xml
        results/**/*.*.pdf

renode-sil:
  runs-on: ubuntu-latest
  needs: matlab-sil
  if: github.event_name == 'push'
  steps:
    - uses: actions/checkout@v4

    - name: Install ARM toolchain
      run: |
        sudo apt-get update
        sudo apt-get install -y gcc-arm-none-eabi cmake

    - name: Build STM32 firmware
      run: |
        mkdir build && cd build
        cmake -DCMAKE_TOOLCHAIN_FILE=../cmake/arm-none-eabi.cmake ..
        make -j4

    - name: Install Renode
      run: |
        wget https://github.com/renode/renode/releases/download/v1.14.0/
          renode_1.14.0_amd64.deb
        sudo dpkg -i renode_1.14.0_amd64.deb

    - name: Run Renode regression
      run: renode -e "include @scripts/regression.resc; quit"
      timeout-minutes: 10

```

14.3 Baseline Karşılaştırması

CI'da kritik nokta, her koşumun sonuçlarının önceki “onaylı” baseline ile karşılaştırılmasıdır. Tipik kabul kriteri: KPI'ların %5'ten fazla sapmaması.

```

1 function pass = compare_against_baseline()
2 % COMPARE_AGAINST_BASELINE Compare current run KPIs against committed
   baseline.
3
4 baseline = load('tests/baseline_kpis.mat');
5 current = load_current_kpis();
6
7 tolerances = struct( ...
8     'E_net',      0.5, ...      % +/- 0.5 Wh
9     'track_rms',  0.3, ...      % +/- 0.3 deg
10    'move_pct',   3.0, ...      % +/- 3 percentage points
11    'data_loss',  0);          % strict
12
13 fields = fieldnames(tolerances);
14 pass = true;
15
16 fprintf('\n==== REGRESYON KONTROLU ==== \n');
17 for f = 1:numel(fields)

```



```

18     fname = fields{f};
19     diff = abs(current.(fname) - baseline.(fname));
20     if diff > tolerances.(fname)
21         fprintf('FAIL   %s: %.3f vs baseline %.3f (diff %.3f > tol %.3f)\n',
22             ...
23             fname, current.(fname), baseline.(fname), ...
24             diff, tolerances.(fname));
25         pass = false;
26     else
27         fprintf('PASS   %s: %.3f (within +/- %.3f)\n', fname, current.(fname),
28             ...
29             tolerances.(fname));
30     end
31 end
32 if ~pass
33     error('REGRESYON DETEKTORU: KPI sapmalari toleransi asti.');
```

Listing 44: compare_against_baseline.m — regresyon dedektörü.

14.4 Baseline Güncelleme Politikası

Baseline'ı ne zaman güncellemeli?

- **Asla:** bug fix'lerinde — bug fix sonuçları değiştirmeli, baseline değişmemeli.
- **Bilerek:** algoritma iyileştirmesinde — yeni baseline'ı PR'da explicit commit'le.
- **Otomatik değil:** CI baseline'ı kendi başına asla güncellememeli.

14.5 Bildirim ve Görselleştirme

CI'dan gelen sonuçlar takılı kalmamalı; her gece otomatik rapor üret:

Listing 45: Slack/Telegram bildirim script'i.

```

# .github/workflows/notify.yml (excerpt)
- name: Slack notification
  if: failure()
  uses: 8398a7/action-slack@v3
  with:
    status: ${ job.status }
    text: |
      SIL regression FAILED on commit ${ github.sha }.
      Run: ${ github.run_id }
      KPI deltas in artifacts.
    webhook_url: ${ secrets.SLACK_WEBHOOK }
```

15 Donanım Bring-up ve Faz 6 Detayı

SIL bittikten ve hipotez doğrulandıktan sonra fiziksel donanıma geçiş, dokümanın orijinal v2.0'daki Faz 6'sının detaylı uygulamasıdır. Bu bölüm SIL ile fiziksel sistem arasındaki köprüyü kurar.

15.1 Pre-Installation Doğrulama Kontrol Listesi

Saha kurulumuna gitmeden önce 24 saat öncesi yapılacaklar:

Table 12: Saha çıkışı öncesi 24-saat kontrol listesi.

Kontrol	Geçti?	Aksiyon
SIL Aşama 3 son koşumu pass	☐	24h içinde olmalı
Firmware SHA hash'ı kaydedildi	☐	repo'da tag, README'de hash
Batarya %100 şarj edildi	☐	multimetre ile $V \geq 13.4 V$
Tüm INA226'lar kalibre edildi	☐	multimetre vs INA, $< \%2$ sapma
4 LDR fonksiyonel test	☐	el feneri ile her birine yön ver
4 DS18B20 adresleri kaydedildi	☐	oda sıcaklığında okuma alınmalı
RTC zamanı senkronize	☐	PC saatiyle eşitleme
SD kart formatlandı (FAT32)	☐	boş, test dosyası yazılıp silindi
IP65 conta-rakor kontrolü	☐	gözle ve hafif çekme testi
Worm gear yağlandı	☐	gres veya silikon sprey
Yedek SD kart + bataryası	☐	çantada hazır
Multimetre + termometre çantada	☐	alan kalibrasyonu için
Laptop + USB kablo + UART köprü	☐	canlı veri için
WiFi/4G erişim noktası	☐	uzak debug için

15.2 Mekanik Montaj Sırası

1. **Sehpa kurulumu:** 3 ayaklı ahşap veya çelik sehpa, kuzeyle hizalı (azimut 0°). Düzleme kontrol için sıvı tesviye.
2. **Pan motor montajı:** Worm-gear DC motor, düşey eksen etrafında dönecek. Limit switch konumlandırma: -90° ve $+90^\circ$ uç noktalar.
3. **Tilt motor montajı:** Yatay eksen, 0° (yatay) – 90° (zenit).
4. **Panel plakası:** 4 paneli barındıracak alüminyum plaka. Petal hinges sol/sağ.
5. **LDR septum mount:** Plakanın tam ortasında, 4 LDR piramidal yapıda. h ve s değerleri kalibrasyon sonrası kesinleşmiş olmalı.
6. **DS18B20 sensör montajı:** 4 panelin arkasına, termal macunla yapıştır, Kapton bantla sabitle.
7. **Kutu mounting:** IP65 polikarbonat kutu, sehpanın gölge altındaki bölümünde (kutu sıcaklığı $< 50^\circ C$ kalmalı).

15.3 Elektriksel Bağlantı Sırası — Asla Tek Adımda Tüm Sistemi Bağlama!

Bring-up'ta her birim ayrı ayrı test edilmeli. Tek seferde tüm sistemi bağlamak, bir hatanın yerini tespit etmeyi imkansızlaştırır.

1. **Adım 1 (sadece batarya):** Batarya + Pololu buck. Çıkış voltajını ölç. $5\text{ V} \pm 0.1$ olmalı.
2. **Adım 2 (sadece mikrodenetleyiciler):** Buck'ı STM32 ve ESP32'ye bağla. LED blink kontrolü.
3. **Adım 3 (motor sürücüsü):** BTS7960'ı bataryaya doğrudan bağla. STM32'den manuel PWM ver. Worm-gear self-locking testi: PWM=0 iken motor hareketsiz mi?
4. **Adım 4 (LDR'ler):** ADC pinlerine LDR'leri bağla. Aydınlık/karanlık değerleri logla.
5. **Adım 5 (INA226 zinciri):** Her INA226'yı I2C'ye birer birer ekle. Adres çakışması kontrolü.
6. **Adım 6 (DS18B20):** 1-Wire bus'a sırayla 4 sensör ekle. Adresleri kaydet.
7. **Adım 7 (microSD):** ESP32'ye bağla, test dosyası yaz/oku.
8. **Adım 8 (UART köprü):** STM32 ve ESP32 arasındaki RX/TX bağla. Heartbeat testi.
9. **Adım 9 (paneller):** 4 PV paneli MPPT girişine bağla. Açık-devre voltajını ölç.

15.4 First Boot Doğrulaması

Sistemi ilk kez gerçek panellerle çalıştırırken yapılacak gözlem:

Table 13: First-boot yaşam belirtileri.

Olay	Ne zaman?	Gözlem
ESP32 boot mesajı	$t = 0$	USB seri konsoldan: "ESP32 logger ready"
STM32 ilk heartbeat	$t < 5\text{ s}$	UART logundan "HB STM32"
RTC zaman doğrula	$t < 10\text{ s}$	Saat PC ile eşitlenmiş mi?
İlk SD yazma	$t \approx 60\text{ s}$	SD'de ilk CSV satırı oluşmalı
İlk MOVE komutu	İlk LDR fark $> \epsilon_{db}$	Motor sesi, paneller dönmeli
Watchdog reset?	İlk saat	0 olmalı

15.5 On-Site LDR Kalibrasyonu (Bölüm 5 ile bağlantı)

Sahada SIL modeli ile gerçek arasındaki en kritik fark LDR davranışdır. Saha kurulumunun ilk gününde mutlaka kalibrasyon koşumu yap:

1. Sehpayı manuel olarak kuzeye 0° pan, 30° tilt'e ayarla (lokal güneş ortalama yüksekliği).
2. Güneşin gün boyu hareketini takip et (manuel, RTC ile zaman damgalı).
3. Her 15 dakikada 4 LDR ADC değerlerini logla.
4. Eş zamanlı güneş efemerisi modelinden gerçek pan/tilt'i hesapla.
5. Saha verisini `ldr_calibration.m` ile uydur.
6. Bulunan h, s değerlerini Plant_Subsystem'e yansıt, SIL'i yeniden çalıştır.

15.6 Veri Toplama Doğrulama Periyodu

İlk 7 günlük veri toplama döneminde her gün:

- Sabah 08:00: SD kartı çıkar (yedek SD takılı kalır), PC’de oku, CSV bütünlüğü kontrolü.
- 09:00 ve 18:00: USB ile sisteme bağlan, canlı veri akışını gör.
- Akşam: Günlük raporu üret (E_net, FSM %, anomaliler).
- İlk haftadaki günler arasında %20’den fazla varyans varsa kalibrasyon tekrarı.

15.7 Saha Hata Mod ve Etki Analizi (FMEA)

Table 14: Saha FMEA — riskli olaylar ve önleyici tedbirler.

Hata Modu	Olas. ^a	Etki ^b	RPN ^c	Önleyici Tedbir
Kuş pisliği LDR’de	Y	O	12	Haftalık temizlik; UV-koruyucu cam
Şiddetli rüzgar	D	Y	6	Tilt 0°’e geç ($V_{wind} > 15$ m/s)
Yağmur infiltrasyonu	D	Y	6	IP65 gore-tex valve düzenli kontrolü
Sıcak çarpma (kutu)	O	Y	12	Fan + gölgelik; $T > 60^{\circ}\text{C}$ ’de SAFE
SD kart bozulma	O	O	9	Industrial-grade SD; günlük rotasyon
Batarya derin deşarj	D	Y	6	$V < 11.5$ V’da derhal SAFE
Hayvan müdahalesi	O	D	6	Tel örgü çerçeve
İnsan müdahalesi	D	O	4	Asma kilit; uyarı tabelası

a: Y=Yüksek, O=Orta, D=Düşük. b: Aynı skala. c: Risk Priority Number = $\text{Olas.} \times \text{Etki} \times \text{Tespit zorluğu}$.

15.8 SIL→Saha Çapraz Referans

Her bir SIL bileşeninin gerçek donanım karşılığı:

16 Sorun Giderme Rehberi (Troubleshooting)

SIL geliştirme sürecinde sık karşılaşılan hatalar ve çözüm yolları. Bu bölüm yıllarca süren prototip çalışmalarının damıtılmış birikimini içerir.

16.1 Aşama 1 (MATLAB / Simulink) Sorunları

16.1.1 “Algebraic loop detected”

Belirti: Simulink koşumu başlatıldığında “Algebraic loop” hatası verir.

Sebebi: Plant’tan Controller’a, Controller’dan Plant’a anlık (sıfır gecikmeli) geri besleme yolu var. Simulink bu döngüyü çözemez.

Çözüm: Controller çıkışı (PWM) ile Plant girişi arasına bir adet **Unit Delay** (z^{-1}) bloğu ekle. Fiziksel olarak da ADC örnekleme + PWM güncelleme arası bir tick gecikme vardır, dolayısıyla model daha gerçekçi olur.

Table 15: SIL bileşenleri vs. gerçek donanım çapraz referansı.

SIL Bileşeni	Gerçek Donanım	Doğrulama Yöntemi
solar_ephemeris_local	GPS modülü (opsiyonel) veya RTC + sabit konum	GPS log vs hesaplanan
ldr_septum_model	4× LDR + 3D-baskılı septum	Bölüm 5 kalibrasyonu
panel_iv_model	4× 5W PV panel	INA226 #1–4 saha verisi
faiman_temp	4× DS18B20	Saha sıcaklık log'u
motor_step	DC motor + worm gear	Tachometer veya step counting
parasitic_power_model	INA226 #6 (system bus)	Saha güç logu
battery_model	LiFePO4 4S 7Ah	Batarya yönetim devresi
Stateflow FSM	STM32 firmware	Equivalence test (Bölüm 9)
UART simülasyonu	Gerçek RX/TX kabloları	Wireshark + UART logger
SD blokaj simülasyonu	Gerçek microSD + FAT32	ESP32 yazma süre log'u

16.1.2 “Stateflow chart yields wrong output type”

Belirti: Stateflow chart çıkışı `double` dönerken Plant bloğu `int16` bekliyor; “port type mismatch” uyarısı.

Çözüm: Chart properties → Symbol Manager → ilgili çıkışın **Type**'ını `int16` olarak ayarla. Veya Plant girişine **Data Type Conversion** bloğu ekle.

16.1.3 “FSM hiç MOVE durumuna girmiyor”

Belirti: 24 saatlik koşulda LDR'lar gözle görülür şekilde dengesiz olduğu halde FSM hep SLEEP–WAKE–DECIDE–REPORT çevriminde kalıyor.

Olası Sebepler ve Çözümler:

1. **Deadband çok büyük:** ϵ_{db} değerini düşür (örn. 50).
2. **LDR voltaj farkı sifıra yakın:** `ldr_septum_model`'de h/s oranı çok küçük olabilir. Yüksekliği artır.
3. **ADC quantization:** 12-bit ADC $LSB \approx 0.8$ mV. Eğer septum geometrisi 1 mV'den küçük fark üretiyorsa, ADC gürültüsü içinde kaybolur. Septum geometrisini büyüt veya LDR direnci bölücüsünü ayarla.

16.1.4 “parsim çok yavaş”

Belirti: 30 koşumluk parsim 5 saatten fazla sürüyor.

Çözüm: (a) Solver tipini `ode4` yerine `ode1` (Euler) yap (Plant zaten 10 ms steps'te lineer). (b) `UseFastRestart=true` ekle. (c) Logging'i `logsout` yerine sadece son özet metriklere indir. (d)

Parallel Computing Toolbox açıkken `parpool('local', 8)` çağır.

16.2 Aşama 2 (Renode + Wokwi) Sorunları

16.2.1 “Renode .elf yükleyemiyor”

Belirti: `sysbus LoadELF` “Image segment too large” hatası.

Çözüm: Linker script’te `FLASH` bölgesi 128 KB ile sınırlı olduğundan, derleyici optimizasyonunu `-Os` yap. Veya `STM32G431KB` yerine `G431RB` seç (256 KB).

16.2.2 “Wokwi UART verisi gelmiyor”

Belirti: Renode’da `showAnalyzer usart1` ile veri akışı görülüyor ama Wokwi tarafında ESP32 hiç frame okumuyor.

Çözümler:

1. Port çıkışması: `netstat -tlnp | grep 6000` ile başka programın aynı portu dinlemediğinden emin ol.
2. `wokwi.toml`’da `forward = ["uart1:tcp:6000"]` satırının doğru pinleri işaret ettiğinden emin ol (UART1 vs UART0).
3. Baud rate: Renode tarafı 115200, Wokwi tarafı da 115200 olmalı. ESP32 firmware’inde `uart_param_config()` ayarını kontrol et.

16.2.3 “ADC bridge plugin yüklenmedi”

Belirti: Renode konsoluna `include @adc_bridge.py` yazılınca “cannot find machine” veya benzeri hata.

Çözüm: Plugin’i `mach create` ve `LoadELF` komutlarından *sonra* yükle. Renode `machine` değişkeni ancak bu noktada tanımlı olur.

16.3 Aşama 3 (Co-Simulation) Sorunları

16.3.1 “Senkronizasyon bozuk: MATLAB çok hızlı”

Belirti: MATLAB plant simülasyon zamanı saniyede 100 simülasyon saniyesi ilerliyor, ama Renode/Wokwi real-time çalışıyor. Senkronizasyon kayboluyor.

Çözüm: `Simulation` → `Pacing` ayarını $1\times$ yap. Plant’a `pause(dt)` blok’u ekle. Daha iyisi: `external mode`’da çalıştır ve sistem clock’una göre tetiklen.

16.3.2 “TCP retry storm”

Belirti: Bir TCP bağlantı kopunca diğer tüm bağlantılar yeniden bağlanmaya çalışıyor, sistem dakikalarca cevap vermiyor.

Çözüm: Her TCP server’da `SO_REUSEADDR=1` set et. Client tarafında exponential backoff retry kullan (1 s, 2 s, 4 s, ...).

16.3.3 “Co-simulation’da E_{net} Aşama 1’den çok düşük”

Belirti: Aşama 1’de $E_{\text{net}} = +6.2 \text{ Wh}$, Aşama 3’te $E_{\text{net}} = -1.5 \text{ Wh}$.

Olası Sebepler:

1. Firmware’de uyku moduna geçişin doğru çalışmadığı (STM32 sürekli aktif kalıyor).
2. UART gecikme MOVE süresini uzatıyor (PWM stopped zamanı geciken raporlama).
3. Watchdog tetiklenip sistemi resetliyor (her reset 100–200 ms aktif zaman).

Tanılama: ESP32 log’undaki FSM state dağılımına bak. MOVE% Aşama 1’de %17 idi, Aşama 3’te %40’a çıktıysa firmware’de bir bug var.

16.4 Donanım/Saha Geçiş Sorunları (Önleyici)

16.4.1 “INA226 sapması Aşama 1’le tutmuyor”

Önleyici tedbir: Sahada her INA226’yı kalibre eden bir prosedür yaz:

Listing 46: INA226 kalibrasyon yordamı.

```
1. Multimetre ile shunt direncini olc (4-wire kelvin).
2. R_meas ile etiketteki R_label arasında > %2 sapma varsa:
   a. INA226’nin CALIBRATION register’ini guncelle
      CAL = round(0.00512 / (Current_LSB * R_shunt))
3. Bilinen yuk (10 ohm pasif diren) bagla, multimetre ile akimi olc.
4. INA226 okumasini multimetreyle karsilastir, %1 icinde olmalı.
```

16.4.2 “Saha akşam: petal kapanmıyor”

Olası Sebep: Solar paneller petal motorunu beslediği için, akşam ışıık düştüğünde sistem gücü düşer ve mikroişlemci reset olur.

Önleyici: Petal kapanma kararı *batarya* ile beslensin, panel değil. Ayrıca akşam saatlerinden 30 dk önce (RTC ile programlı) petal kapatma komutu verilsin — LDR’ye bağımlı bırakma.

16.5 Genel Debug Stratejisi: “Divide-and-Conquer”

Bir hata izlenirken sistematik yaklaşım:

1. **Hata reproduksiyonu:** Hatanın deterministik olarak tekrarlandığı en küçük senaryoyu izole et.
2. **Yarıyı kapat:** Şüphe duyulan iki bileşenden birini devre dışı bırak (**stub** ile). Hata kalıyor mu?
3. **Loglama artır:** İlgili noktaya **printf** veya signal logging ekle.
4. **Hipotez kur:** “X yüzünden Y oluyor” önerme yaz.
5. **Test et:** X’i değiştir ve Y’nin değişmesini bekle.

6. **Doğrula:** Hipotez doğrulanırsa kalıcı düzeltme yap. Değilse başka hipotez kur ve adım 4'e dön.

Bu yaklaşım gömülü sistemlerde özellikle önemlidir; gözlemcinin sisteme dokunması ("probe effect") bazı hataları değiştirir, dolayısıyla sistematik olmadan hatayı koalamak günler kaybettirir.

17 Akademik Yazım ve Veri Sunumu

SIL'den elde edilen sonuçların tezde / makalede en yüksek değeri taşıması için nasıl raporlanacağı önemlidir. Bu bölüm yapılandırılmış öneriler sunar.

17.1 Tez Bölümü Önerilen Yapısı

SIL çalışması, deneysel yöntem (Bölüm 3) ve sonuçlar (Bölüm 4) içinde şu yapıda sunulmalıdır:

- **Bölüm 3.1 — Sistem Tasarımı:** Donanım (orijinal v2.0 dokümanı).
- **Bölüm 3.2 — Yazılım Tasarımı:** FSM, PI, UART protokolü.
- **Bölüm 3.3 — Doğrulama Metodolojisi:** V-modeli, SIL aşamaları, bu dokümanın özeti.
- **Bölüm 3.4 — Plant Modeli:** Güneş efemerisi, panel I-V, Faiman.
- **Bölüm 4.1 — SIL Birim Test Sonuçları:** Coverage tabloları.
- **Bölüm 4.2 — SIL Sistem Test Sonuçları:** 24 saatlik gün örnekleri.
- **Bölüm 4.3 — Parametre Duyarlılık:** Tornado, Pareto.
- **Bölüm 4.4 — Hipotez Testi:** 30-koşum istatistik.
- **Bölüm 4.5 — Saha Karşılaştırması:** MIL/SIL/Saha üçlü grafik.

17.2 Numerik Sonuçların Sunumu

Akademik rapor yazımında numerik değerler için temel kurallar:

- **Anlamli basamak:** Net enerji 4.27 ± 0.31 Wh; ham sayı yerine belirsizlikle birlikte yaz.
- **p-değeri:** " $p < 0.05$ " yeterli değil; tam değer ver (" $p = 0.0028$ " gibi).
- **Güven aralığı:** Tek değer yerine %95 CI ver: " $E_{\text{net}} \in [3.65, 4.89]$ Wh".
- **Efekt boyutu:** İstatistiksel anlam $\neq$ pratik anlam. Cohen's d veya benzeri hesapla.

17.3 Şekil Yazım Standardı

Her şekilde olmazsa olmaz öğeler:

1. Açıklayıcı caption (figür yalnız okunsa bile anlaşılır olmalı).
2. Her eksen birimli etiket ("Time [s]", "Power [W]").

3. Eğri başına label (legend) veya doğrudan etiketleme.
4. Belirsizlik bandı (1σ veya CI95) çoklu koşulda.
5. Referans çizgileri (hedef, eşik) dashed gri.

17.4 Tablo Yazım Standardı

- `booktabs` paketi kullan (`\toprule`, `\midrule`, `\bottomrule`).
- Vertical line kullanma.
- Sayıları decimal point hizala.
- Birim'i sütun başlığında parantez içinde ver: "Power (W)".
- Karşılaştırma tablosunda en iyi değeri **bold** yap.

17.5 Reproducibility Checklist

Akademik makalenin reprodüce edilebilir olması için ek raporlama:

- MATLAB sürümü ve toolbox listesi.
- Renode commit hash.
- Wokwi CLI sürümü.
- Tüm parametrelerin tablosu (Tablo 6 stili).
- TMY veri kaynağı (PVGIS doi'si).
- Kaynak kod public repo (GitHub/Zenodo doi).
- Ham veri set archive (Mendeley Data veya kurumsal repo).

17.6 Hipotez Yanlış Çıkarsa Ne Yazarsın?

Tez savunmasında en sık sorulan soru: "Eğer $E_{\text{net}} < 0$ çıktıysa proje başarısız mı?" Cevap kesinlikle hayır. Şu şekilde sun:

"Yapılabilirlik analizinde gerçekçi senaryoda +2.5 Wh marjla başlayan hipotez, firmware gecikmeleri ve gerçekçi LDR gürültüsü modellendiğinde $E_{\text{net}} = -0.8 \pm 0.5$ Wh'a düşmüştür. Bu, küçük ölçekli (< 50 W) takip sistemlerinin parazitik bütçesini aşmasının ne kadar dar bir marja bağlı olduğunu gösteren ilk SIL doğrulamalı çalışmadır. Literatürdeki tipik > 1 kW prototipler bu sınırı geçer, ancak ölçeklendirme her zaman doğrusal değildir."

Yani *negatif sonuç bile bilime katkıdır*, özellikle de gerekçesi tarafsız ölçümlerle desteklenmişse.

18 Akademik Değer ve Sonraki Adımlar

18.1 Tez/Makalede SIL'in Yeri

Bu SIL çalışmasının asıl katkısı, küçük ölçekli (<50 W) güneş takip sistemleri için literatürde nadir görülen **üçlü doğrulama zinciri**dir:

1. **MIL (Model-in-the-Loop)**: Sadece teorik denklemler, sonsuz hassasiyet varsayımı.
2. **SIL (Software-in-the-Loop)**: Algoritma + protokol davranışı, yazılım gerçekçiliği.
3. **HIL/Saha**: Donanım toleransları, çevresel gürültü, gerçek hava verisi.

Şekil 3'teki gibi bir grafik tez savunmasında veya dergi makalesinde “hipotez ne kadar dirençli?” sorusunun cevabını net görsellerle verir.

18.2 Bu Plana Bağlı Yan Çalışmalar

- **Parametre duyarlılık analizi**: Aşama 1 hazırlarken ϵ_{db} , K_p , K_i , sleep süresi gibi parametreleri taramak (parsim ile paralel).
- **LDR gürültü modellemesi**: Gerçek LDR'lerin ADC gürültü profilini ölçmüş olmak ve onu Plant_Subsystem'a beyaz/pembe gürültü olarak eklemek.
- **Bulut modeli**: Markov zinciri tabanlı sentetik bulut serisi ile MIL'deki sürekli güneş varsayımını test etmek.

18.3 Riskler ve Azaltmaları

Table 16: SIL geliştirme aşamasındaki riskler.

Risk	Olası.	Azaltma
Mevcut MATLAB kodunun Simulink imzasına uymaması	Orta	Önce küçük bir wrapper test'i (Bölüm 4.3'teki iskelet) ile uyumu kanıtla.
Stateflow'da CRC hesabının kodgen hatası	Düşük	CRC fonksiyonunu MATLAB Function'a koy, Stateflow'da çağır.
Renode'da STM32G4 platform desteginde eksiklik	Orta	Stm32G4 .repl dosyasını gerekirse manuel düzenle veya STM32F4 fallback.
Wokwi'de DS3231 sürüklenmesi	Düşük	RTC değerini ESP32 firmware'inde sntp benzeri kaynaktan al.
TCP latency gerçek seri portu temsil etmiyor	Yüksek	TCP send sonrası Transport Delay bloğu ile UART byte süresini ekle.

18.4 Sonraki Adım (Bu Doküman Sonrası)

1. Bu dokümanı oku, Bölüm 4'in tamamını işle.
2. Yeni bir Simulink modeli (solar_tracker_SIL.slx) oluştur.
3. Mevcut MATLAB fonksiyonlarını Plant_Subsystem içine taşı.

4. Stateflow FSM'ini oluřtur (Bölüm 4.4'teki pseudocode'a göre).
5. İlk kořumu yap: 5 dakikalık sabit güneř senaryosu.
6. İlk debug ve geri-bildirim için sistem dokümanı yazarına dön.

A Ek A: Tam Plant_Subsystem MATLAB Function Şablonu

```

1 function [ldr_adc, P_panel, V_panel, I_panel, P_mppt, P_bus, ...
2         T_panel, sun_az, sun_el, panel_pan_out, panel_tilt_out] = ...
3         plant_step(pwm_pan, pwm_tilt, t_sim, GHI, T_amb, wind)
4 %codegen
5 % PLANT_STEP Solar tracker plant model (single Simulink time step).
6 %
7 % INPUTS
8 %   pwm_pan, pwm_tilt : uint16, [-1000, +1000], duty * 1000
9 %   t_sim             : double, sec from sim start
10 %   GHI               : double, W/m^2 (global horizontal)
11 %   T_amb             : double, deg C
12 %   wind              : double, m/s
13 %
14 % OUTPUTS
15 %   ldr_adc(4)        : uint16, 0-4095 (12-bit ADC)
16 %   P_panel(4), V_panel(4), I_panel(4): double per panel
17 %   P_mppt, P_bus     : double, W (system production / parasitic)
18 %   T_panel(4)        : double, deg C (Faïman model)
19 %   sun_az, sun_el    : double, deg
20 %   panel_pan_out     : double, deg (current panel orientation)
21 %   panel_tilt_out    : double, deg
22
23 persistent panel_pan panel_tilt motor_pan_w motor_tilt_w
24 if isempty(panel_pan)
25     panel_pan = 0; panel_tilt = 30; % kuzey-yari kuresi varsayim
26     motor_pan_w = 0; motor_tilt_w = 0;
27 end
28
29 dt = 0.01; % must match Simulink fundamental sample time
30
31 % ---- (1) Solar ephemeris ----
32 % TODO: mevcut MATLAB kodunu cagır
33 [sun_az, sun_el] = solar_ephemeris_local(t_sim); % deg
34
35 % ---- (2) LDR via septum geometry ----
36 mis_pan = sun_az - panel_pan;
37 mis_tilt = sun_el - panel_tilt;
38 ldr_v = ldr_septum_model(mis_pan, mis_tilt, GHI); % 4x1 [V]
39 ldr_adc = uint16( min(4095, max(0, round(ldr_v*4095/3.3))) );
40
41 % ---- (3) Panel IV & thermal ----
42 [P_panel, V_panel, I_panel] = panel_iv_model(mis_pan, mis_tilt, GHI, T_amb);
43 T_panel = faïman_temp(GHI, T_amb, wind, [1;1;1;1]);
44 P_mppt = sum(P_panel);
45
46 % ---- (4) Parasitic budget ----
47 P_bus = parasitic_power_model(t_sim, pwm_pan, pwm_tilt);

```

```

48
49 % ---- (5) Motor dynamics (2nd order) ----
50 J = 0.005;    B = 0.05;    Ktau = 2e-5;    % NOTE: tune from real motor
51 % pan
52 tau_pan = Ktau * double(pwm_pan);
53 motor_pan_w = motor_pan_w + dt*(tau_pan - B*motor_pan_w)/J;
54 panel_pan = panel_pan + dt*rad2deg(motor_pan_w);
55 panel_pan = max(-90, min(90, panel_pan));    % limit switch
56 % tilt
57 tau_tilt = Ktau * double(pwm_tilt);
58 motor_tilt_w = motor_tilt_w + dt*(tau_tilt - B*motor_tilt_w)/J;
59 panel_tilt = panel_tilt + dt*rad2deg(motor_tilt_w);
60 panel_tilt = max(0, min(90, panel_tilt));
61
62 panel_pan_out = panel_pan;
63 panel_tilt_out = panel_tilt;
64 end

```

Listing 47: Plant_Subsystem.m — tam şablon.

B Ek B: Stateflow Chart Kurulumu Adımları

1. Simulink'te STM32_Controller subsystem'ı aç.
2. Library Browser → Stateflow → Chart'ı çift tıkla.
3. Chart'ı subsystem'a sürükle.
4. Chart'ı çift tıkla, 7 state oluştur: INIT, SLEEP, WAKE, DECIDE, MOVE, REPORT, SAFE.
5. Default geçişi INIT'e bağla.
6. Input/Output portlarını Symbols panelinden ekle (Bölüm 4.4'teki imzaya göre).
7. Her state'in entry/during/exit eylemlerini Bölüm 4.4'teki pseudocode'a göre doldur.
8. PI fonksiyonunu (pi_step.m) Chart'a Graphical Function olarak ekle veya ayrı MATLAB Function bloğu olarak Chart dışına koy.
9. Chart properties'te Update method = Discrete, Sample Time = 0.05.
10. Sırasıyla küçük testler:
 - Sabit LDR girişi → FSM uyanma döngüsünü scope ile gözle.
 - Dengeli LDR → MOVE'a girmemeli.
 - Dengesiz LDR → MOVE → MOVE içinde error sıfıra düşmeli → REPORT.

C Ek C: Tam Stateflow Chart İçeriği

Aşağıda Aşama 1'deki STM32_Controller içindeki Stateflow chart'ının tam Stateflow Editor pseudocode hali yer almaktadır. Her bir state'in entry, during, exit eylemleri ve geçiş koşulları ayrı

ayrı verilmiştir.

C.1 Chart Properties

- Action Language: MATLAB
- Update Method: Discrete
- Sample Time: 0.05 (50 ms)
- Enable C-bit operations: ON
- Strong data typing: ON

C.2 Chart Inputs/Outputs/Locals

Listing 48: Stateflow Symbol panel tanımları.

```
INPUTS:
  ldr_adc      : uint16 [4]      (12-bit ADC values)
  vbat         : single          (battery voltage)
  emergency    : boolean         (external alarm trigger)

OUTPUTS:
  pwm_pan      : int16
  pwm_tilt     : int16
  fsm_state    : uint8
  uart_frame_out : uint8 [22]
  report_trigger : boolean

LOCALS (persistent):
  integral_pan  : single = 0
  integral_tilt : single = 0
  e_pan         : int32
  e_tilt        : int32
  sleep_until   : single = 0
  move_timer    : single = 0
  move_start_time : single = 0

CONSTANTS:
  DEADBAND      : int32 = 100
  KP             : single = 0.5
  KI             : single = 0.1
  TS            : single = 0.05
  IMAX          : single = 500
  UMAX          : int32 = 1000
  STALL_TIMEOUT_S : single = 3.0
  SLEEP_DURATION_BRIGHT : single = 30
  SLEEP_DURATION_CLOUDY : single = 120
  SLEEP_DURATION_NIGHT  : single = 600
```

C.3 State: INIT

```
entry:
  integral_pan = 0;
```

```

    integral_tilt = 0;
    pwm_pan      = int16(0);
    pwm_tilt     = int16(0);
    fsm_state    = uint8(1);
    sleep_until  = 0;

during:
    % init self-test complete after one tick

exit:
    % no action

```

Geçiş: INIT → SLEEP default’a basıldıktan sonra koşulsuz.

C.4 State: SLEEP

```

entry:
    pwm_pan      = int16(0);
    pwm_tilt     = int16(0);
    fsm_state    = uint8(2);
    % Adaptif sleep suresi: gunduz parlaksa kısa, bulutluda orta, gece uzun
    ldr_mean     = mean(ldr_adc);
    if ldr_mean > 800
        sleep_until = t + SLEEP_DURATION_BRIGHT;
    elseif ldr_mean > 200
        sleep_until = t + SLEEP_DURATION_CLOUDY;
    else
        sleep_until = t + SLEEP_DURATION_NIGHT;
    end

during:
    % Stop Mode’da olmayi simule et: hicbir is yapma

exit:
    % no action

```

Geçişler:

- SLEEP → WAKE [koşul: $t \geq \text{sleep_until}$]
- SLEEP → SAFE [koşul: $\text{emergency} == \text{true}$]

C.5 State: WAKE

```

entry:
    fsm_state = uint8(3);
    % LDR snapshot al
    % (Plant_Subsystem zaten ldr_adc’yi guncelliyor, ekstra is yok)

during:
    % no action

exit:
    % no action

```

Geçiş: WAKE $\rightarrow$ DECIDE koşulsuz, bir tick sonra.

C.6 State: DECIDE

```
entry:
    fsm_state = uint8(4);
    e_pan = int32(int32(ldr_adc(1)) + int32(ldr_adc(3))) - ...
            int32(int32(ldr_adc(2)) + int32(ldr_adc(4)));
    e_tilt = int32(int32(ldr_adc(1)) + int32(ldr_adc(2))) - ...
            int32(int32(ldr_adc(3)) + int32(ldr_adc(4)));

during:
    % no action

exit:
    % no action
```

Geçişler:

- DECIDE $\rightarrow$ REPORT [koşul: $\text{abs}(e_{\text{pan}}) < \text{DEADBAND} \ \&\& \ \text{abs}(e_{\text{tilt}}) < \text{DEADBAND}$]
- DECIDE $\rightarrow$ MOVE [varsayılan diğer durumlarda]
- DECIDE $\rightarrow$ SAFE [koşul: $\text{vbat} < 10.0$]

C.7 State: MOVE

```
entry:
    fsm_state = uint8(5);
    move_start_time = t;
    move_timer = 0;

during:
    % PI step for pan
    integral_pan = integral_pan + single(e_pan) * TS;
    if integral_pan > IMAX, integral_pan = IMAX; end
    if integral_pan < -IMAX, integral_pan = -IMAX; end
    u_pan = KP*single(e_pan) + KI*integral_pan;
    if u_pan > single(UMAX), u_pan = single(UMAX); end
    if u_pan < -single(UMAX), u_pan = -single(UMAX); end
    pwm_pan = int16(u_pan);

    % PI step for tilt
    integral_tilt = integral_tilt + single(e_tilt) * TS;
    if integral_tilt > IMAX, integral_tilt = IMAX; end
    if integral_tilt < -IMAX, integral_tilt = -IMAX; end
    u_tilt = KP*single(e_tilt) + KI*integral_tilt;
    if u_tilt > single(UMAX), u_tilt = single(UMAX); end
    if u_tilt < -single(UMAX), u_tilt = -single(UMAX); end
    pwm_tilt = int16(u_tilt);

    move_timer = t - move_start_time;

    % Refresh error from latest ldr_adc
    e_pan = int32(int32(ldr_adc(1)) + int32(ldr_adc(3))) - ...
```

```

        int32(int32(ldr_adc(2)) + int32(ldr_adc(4)));
    e_tilt = int32(int32(ldr_adc(1)) + int32(ldr_adc(2))) - ...
        int32(int32(ldr_adc(3)) + int32(ldr_adc(4)));

exit:
    pwm_pan = int16(0);
    pwm_tilt = int16(0);

```

Geçişler:

- MOVE → REPORT [koşul: `abs(e_pan) < DEADBAND && abs(e_tilt) < DEADBAND`]
- MOVE → SAFE [koşul: `move_timer > STALL_TIMEOUT_S`]

C.8 State: REPORT

```

entry:
    fsm_state = uint8(6);
    report_trigger = true;
    % Build UART frame and emit
    uart_frame_out = build_uart_frame(panel_pan, panel_tilt, ...
                                     ldr_adc, fsm_state, ...
                                     petal_state, vbat);

during:
    report_trigger = false;

exit:
    % no action

```

Geçiş: REPORT → SLEEP koşulsuz, bir tick sonra.

C.9 State: SAFE

```

entry:
    fsm_state = uint8(7);
    pwm_pan = int16(0);
    pwm_tilt = int16(0);
    % Log alarm event (handled in ESP32 side)

during:
    % manuel reset bekle

exit:
    integral_pan = 0;
    integral_tilt = 0;

```

Geçiş: SAFE → INIT [koşul: `emergency == false && vbat > 11.0`]

D Ek D: Post-Process Boru Hattı

Aşama 1 simülasyonu tamamlandıktan sonra çalıştırılacak tam post-process script:


```

1 function run_postprocess(slx_results, output_dir)
2 % RUN_POSTPROCESS Full analysis pipeline for one SIL simulation run.
3
4 if nargin < 2, output_dir = './results'; end
5 if ~exist(output_dir, 'dir'), mkdir(output_dir); end
6
7 % ===== 1) Workspace verilerini cek =====
8 t = slx_results.logout.getElement('time').Values.Data;
9 P_mppt = slx_results.logout.getElement('P_mppt').Values.Data;
10 P_bus = slx_results.logout.getElement('P_bus').Values.Data;
11 state = slx_results.logout.getElement('fsm_state').Values.Data;
12 pan = slx_results.logout.getElement('panel_pan').Values.Data;
13 tilt = slx_results.logout.getElement('panel_tilt').Values.Data;
14 sun_az = slx_results.logout.getElement('sun_az').Values.Data;
15 sun_el = slx_results.logout.getElement('sun_el').Values.Data;
16
17 % ===== 2) Enerji entegrasyonu =====
18 E_uretim = trapz(t, P_mppt) / 3600;
19 E_parazitik = trapz(t, P_bus) / 3600;
20 E_net = E_uretim - E_parazitik;
21
22 % ===== 3) Takip dogrulugu =====
23 track_err_pan = sun_az - pan;
24 track_err_tilt = sun_el - tilt;
25 track_rms = sqrt(mean(track_err_pan.^2 + track_err_tilt.^2));
26
27 % ===== 4) FSM dagilim =====
28 state_names = {'INIT', 'SLEEP', 'WAKE', 'DECIDE', 'MOVE', 'REPORT', 'SAFE'};
29 pct = zeros(1,7);
30 for s = 1:7
31     pct(s) = 100*sum(state == s)/length(state);
32 end
33
34 % ===== 5) Sonuc raporu =====
35 fid = fopen(fullfile(output_dir, 'run_summary.txt'), 'w');
36 fprintf(fid, '=== SIL Run Summary ===\n');
37 fprintf(fid, 'Simulation length: %.1f h\n', max(t)/3600);
38 fprintf(fid, 'Production: %.2f Wh\n', E_uretim);
39 fprintf(fid, 'Parasitic: %.2f Wh\n', E_parazitik);
40 fprintf(fid, 'NET GAIN: %.2f Wh\n', E_net);
41 fprintf(fid, 'Tracking RMS: %.2f deg\n', track_rms);
42 fprintf(fid, '\n--- FSM time distribution ---\n');
43 for s = 1:7
44     fprintf(fid, ' %-8s : %5.1f %%\n', state_names{s}, pct(s));
45 end
46 fclose(fid);
47
48 % ===== 6) Grafikler =====
49 publication_defaults();
50
51 % Plot 1: Power balance
52 fig1 = figure;
53 subplot(3,1,1); plot(t/3600, P_mppt, 'Color', [0 114 178]/255);
54 ylabel('P_{mppt} [W]'); grid on;
55 subplot(3,1,2); plot(t/3600, P_bus, 'Color', [213 94 0]/255);

```

```

56 ylabel('P_{bus} [W]'); grid on;
57 subplot(3,1,3); stairs(t/3600, state, 'Color', [0 158 115]/255);
58 ylabel('FSM'); xlabel('Time [h]'); grid on;
59 ylim([0 8]); yticks(1:7); yticklabels(state_names);
60 export_publication(fig1, fullfile(output_dir, 'fig_power_balance'), 17.4, 12);
61
62 % Plot 2: Tracking accuracy
63 fig2 = figure;
64 plot(t/3600, sun_az, 'Color', [0 114 178]/255, 'DisplayName', 'Sun Az');
65 hold on;
66 plot(t/3600, pan, '--', 'Color', [213 94 0]/255, 'DisplayName', 'Panel Pan'
67 );
68 plot(t/3600, sun_el, 'Color', [0 158 115]/255, 'DisplayName', 'Sun El');
69 plot(t/3600, tilt, '--', 'Color', [204 121 167]/255, 'DisplayName', 'Panel
70 Tilt');
71 xlabel('Time [h]'); ylabel('Angle [deg]'); legend('Location','best'); grid on;
72 export_publication(fig2, fullfile(output_dir, 'fig_tracking'), 17.4, 8);
73
74 % Plot 3: FSM pie chart
75 fig3 = figure;
76 pie(pct(pct>0), state_names(pct>0));
77 title(sprintf('FSM Distribution (24h)'));
78 export_publication(fig3, fullfile(output_dir, 'fig_fsm_pie'), 8.4, 8);
79
80 % ===== 7) CSV export =====
81 T = table(t, pan, tilt, P_mppt, P_bus, state, sun_az, sun_el);
82 writetable(T, fullfile(output_dir, 'virtual_sd.csv'));
83
84 fprintf('Post-process complete. Results in %s\n', output_dir);
85 end

```

Listing 49: run_postprocess.m — baştan sona analiz boru hattı.

E Ek E: Renode Dosya Şablonları

E.1 solar_tracker.resc Tam İçerik

```

:name: SolarTrackerSTM
:description: STM32G4 solar tracker SIL setup

# --- machine creation ---
mach create "stm32_tracker"
machine LoadPlatformDescription @platforms/cpus/stm32g4.repl

# --- load firmware ---
sysbus LoadELF @solar_tracker.elf

# --- UART1 to TCP 6000 ---
emulation CreateServerSocketTerminal 6000 "stm_uart_external"
connector Connect sysbus.usart1 stm_uart_external

# --- ADC bridge (Python plugin) ---
include @adc_bridge.py

# --- analyzers ---

```

```

showAnalyzer sysbus.usart1
logLevel 1 sysbus.usart1

# --- start in stopped state for debugging ---
machine StartGdbServer 3333 true

# Optional: GPIO observer for PWM output
machine.sysbus.gpioPortA.AddStateChangedObserver 8 \
    "Console.WriteLine(\"PA8 changed at {0}\", machine.ElapsedVirtualTime)"

# Begin simulation
start

```

E.2 Renode Konsol Üzerinden Manuel ADC Set

Plugin yüklü değilse veya manuel debug için:

```

# Renode console:
sysbus WriteDoubleWord 0x50000040 2048 # ADC1->DR = 2048 (~1.65V)

# Veya register tag tanımla:
sysbus Tag <0x50000040, 0x50000044> "ADC1_DR" 0x800

```

E.3 STM32G431KB Platform Açıklaması (.repl)

Renode'da hazır STM32G4 platformu `platforms/cpus/stm32g4.repl` altında bulunur, ancak G431KB'nin spesifik bellek ve çevresel haritası farklı olabilir. Aşağıda en kritik parçalar verilmiştir. Plugin veya custom değişiklik için `user_stm32g431kb.repl` olarak kaydet ve `LoadPlatformDescription` buna yönlendir.

Listing 50: `user_stm32g431kb.repl` — STM32G431KB için Renode platform tanımı (özet).

```

//
// STM32G431KB platform description (excerpt).
// Reference: STMicro RM0440, table 1.
//

using "platforms/cpus/cortex-m4.repl"

// ----- Memory -----
flash: Memory.MappedMemory @ sysbus 0x08000000
    size: 0x20000 // 128 KB

sram1: Memory.MappedMemory @ sysbus 0x20000000
    size: 0x4000 // 16 KB (data)

sram2: Memory.MappedMemory @ sysbus 0x20004000
    size: 0x1800 // 6 KB

ccmram: Memory.MappedMemory @ sysbus 0x10000000
    size: 0x2800 // 10 KB (zero-wait)

// ----- Core / NVIC -----
nvic: IRQControllers.NVIC @ sysbus 0xE000E000

```

```

    priorityMask: 0xF0
    systickFrequency: 170000000 // 170 MHz typical SYSCLK
    IRQ -> cpu@0

cpu: CPU.CortexM @ sysbus
    cpuType: "cortex-m4"
    nvic: nvic
    PerformanceInMips: 170

// ----- Peripherals -----
usart1: UART.STM32F7_USART @ sysbus 0x40013800
    frequency: 170000000
    IRQ -> nvic@37

usart2: UART.STM32F7_USART @ sysbus 0x40004400
    frequency: 170000000
    IRQ -> nvic@38

tim1: Timers.STM32_Timer @ sysbus 0x40012C00
    frequency: 170000000
    initialLimit: 0xFFFFFFFF
    -> nvic@24, nvic@25, nvic@26, nvic@27

tim2: Timers.STM32_Timer @ sysbus 0x40000000
    frequency: 170000000
    -> nvic@28

adc1: Analog.STM32G4_ADC @ sysbus 0x50000000
    referenceVoltage: 3.3
    externalEventFrequency: 0
    IRQ -> nvic@18

i2c1: I2C.STM32F7_I2C @ sysbus 0x40005400
    eventInterrupt -> nvic@31
    errorInterrupt -> nvic@32

spi1: SPI.STM32SPI @ sysbus 0x40013000
    IRQ -> nvic@35

rtc: Timers.STM32F4_RTC @ sysbus 0x40002800
    AlarmIRQ -> nvic@41
    WakeupIRQ -> nvic@3

iwdg: Timers.STM32_IndependentWatchdog @ sysbus 0x40003000
    frequency: 32000
    defaultTimeout: 0xFFFF

gpioPortA: GPIOPort.STM32GPIOPort @ sysbus 0x48000000
    [0-15] -> nvic@[6-10]

gpioPortB: GPIOPort.STM32GPIOPort @ sysbus 0x48000400
    [0-15] -> nvic@[6-10]

sysbus:
    init:
        ApplyTag <0x40021000, 0x40021FFF> "RCC"

```

```
ApplyTag <0x40022000, 0x40022FFF> "FLASH_IF"  
ApplyTag <0x40010000, 0x400103FF> "SYSCFG"
```

E.4 Renode’da Debug Komutları

Faydalı Renode konsol komutları:

```
# CPU durumu  
cpu IsHalted  
cpu PC  
cpu SP  
  
# Memory dump  
sysbus ReadDoubleWord 0x20000100  
sysbus ReadDoubleWord 0x20000100 256 # 256 word dump  
  
# Peripheral register snapshot  
sysbus.usart1 GetRegisters  
  
# Breakpoint  
sysbus.cpu AddHook <pc_address> "Console.WriteLine(\"hit BP\")"  
  
# Step  
machine StepInstruction  
machine Step 100  
  
# Goto-up (single function)  
cpu Step  
cpu StepOver  
cpu StepOut  
  
# Trace and log  
machine EnableLog "/tmp/renode.log"  
logLevel -1 sysbus.usart1 # most verbose  
  
# GDB  
machine StartGdbServer 3333 false # then 'arm-none-eabi-gdb' connect
```

E.5 GDB ile Birleşik Debugging

Renode + GDB kombinasyonu, gerçek STM32 debug deneyimine çok yakındır:

```
# Terminal 1: Renode’u başlat  
renode -e "include @solar_tracker.resc; machine StartGdbServer 3333 true"  
  
# Terminal 2: GDB  
arm-none-eabi-gdb solar_tracker.elf  
(gdb) target remote :3333  
(gdb) break fsm_step  
(gdb) continue  
# ... breakpoint  
(gdb) print integral_pan  
(gdb) backtrace
```

F Ek F: Wokwi Komut Satırı Kullanımı

F.1 Komut Satırı Çalıştırma

```
# Wokwi CLI'yi kur (Node.js gerekli)
npm install -g @wokwi/cli

# Lisans aktive et (ucretsiz seviye saat basına sayilir)
wokwi-cli auth

# Proje dizininde:
ls
# -> diagram.json, wokwi.toml, build/esp32_logger.bin, build/esp32_logger.elf

# Calistir (varsayilan: gercek zaman)
wokwi-cli simulate

# Hizlandirilmis (1 saat sim, 5 dakikada bit)
wokwi-cli simulate --speed 12

# CI/CD icin pythonsuz batch
wokwi-cli test --timeout 60s --expect-text "BOOT_OK"
```

F.2 Wokwi'den UART Loglarini Cekme

```
# Tum UART trafiginin kaydet
wokwi-cli simulate --serial-log uart.log --duration 1h

# Ozet
grep -c "0x10" uart.log      # status frame count
grep -c "CRC_FAIL" uart.log
```

G Ek G: Test Raporu Şablonu

Her test koşumu için doldurulacak rapor şablonu (Markdown formatında). Bu `report.md` dosyaları daha sonra Pandoc ile PDF rapora dönüştürülebilir.

```
# SIL Test Raporu

**Run ID:**      -----
**Tarih:**       YYYY-MM-DD HH:MM
**Operator:**    -----
**Asama:**       [ ] 1   [ ] 2   [ ] 3
**Senaryo:**     -----

## 1. Konfigurasyon

| Parametre          | Deger          |
|-----|-----|
| MATLAB surum       |                 |
| Renode commit      |                 |
| Wokwi CLI surum    |                 |
| Firmware SHA       |                 |
| TMY veri           |                 |
```

```

| Sim suresi | |
## 2. Parametreler

| Param | Deger | Birim |
|-----|-----|-----|
| eps_db | | |
| Kp | | |
| Ki | | |
| Ts | | s |
| sleep_bright | | s |
| sigma_LDR | | % |

## 3. KPI Sonuclari

| KPI | Hedef | Olculen | Gecti? |
|-----|-----|-----|-----|
| E_net | > +2.5Wh | | |
| Tracking RMS | < 3.5 deg | | |
| MOVE % | 10-25% | | |
| Yanlis MOVE tetik | <= 2 | | |
| CRC hata orani | < 0.1% | | |
| Veri kaybi (satir) | 0 | | |
| Watchdog reset | 0 | | |
| Heartbeat timeout | 0 | | |

## 4. Gozlemler ve Anomaliler

(Beklenmeyen davranislar, ozgun olaylar, dikkat cekici grafiksel sapmalar)

## 5. Sonraki Adim

[ ] Bu konfigurasyonla devam et
[ ] Parametreleri yeniden ayarla: -----
[ ] Bug duzelt: -----
[ ] Saha testine gec

```

H Ek H: Sözlük (Glossary)

ADC Analog-to-Digital Converter. Sürekli analog sinyali ayrık dijital değere çeviren modül. Burada 12-bit (0–4095 aralıkta) varsayılmıştır.

Anti-windup

PI kontrolcüde integral teriminin sınırsız büyümesini önleyen saturation mekanizması. Aktüatör doyduğunda integralin daha fazla artmasını engeller.

BOR Brown-Out Reset. Besleme gerilimi belirli bir eşiğin altına düştüğünde MCU’nun otomatik resetlenmesi.

Co-simulation

İki veya daha fazla simülasyon ortamının eşzamanlı çalışıp birbirleriyle veri alışverişi yapması. SIL’in en üst seviyesi.

CRC	Cyclic Redundancy Check. Veri bütünlüğü kontrolü için kullanılan matematiksel checksum.
Deadband	Ölü bölge. Hata sinyali bu eşiğin altındayken kontrol aksiyonunun engellenmesi. Gürültüye karşı dayanıklılık sağlar.
Faiman Model	PV panellerin sıcaklığını çevre koşullarından hesaplayan yarı-ampirik termal model.
FAT32	File Allocation Table 32-bit. SD kartlarda yaygın dosya sistemi.
FSM	Finite State Machine. Sonlu durum makinesi. Sistemin davranışını durumlar ve geçişlerle modelleyen formal yapı.
GHI	Global Horizontal Irradiance. Yatay yüzeye düşen toplam güneş ışıınımı (W/m^2).
HAL	Hardware Abstraction Layer. Donanım soyutlama katmanı. Üst seviye kodun donanım detaylarından bağımsız yazılmasını sağlar.
HIL	Hardware-in-the-Loop. Gerçek donanımın bilgisayar simülasyonu ile döngüde test edilmesi.
IWDG	Independent Watchdog. STM32’de bağımsız donanım watchdog’u.
LDR	Light Dependent Resistor. Işığa duyarlı direnç. Burada güneş pozisyonu tespiti için 4 adet kullanılıyor.
LSB	Least Significant Bit. ADC’nin ayırt edebileceği en küçük adım.
MIL	Model-in-the-Loop. Sistemin tamamen matematiksel modelle simüle edilmesi. SIL’in öncülü.
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor. Burada fan kontrol anahtarı.
MPPT	Maximum Power Point Tracking. PV panelin maksimum güç noktasında çalıştırılması.
Move-and-sleep	Sürekli takip yerine sadece gerektiğinde hareket ettirme felsefesi. Parazitik tüketimi azaltır.
Parsim	MATLAB’in paralel simülasyon koşturma fonksiyonu.
Petal	Bu projede panellerin “çiçek yaprağı” gibi açılıp kapanan mekanizması.
PI Controller	Proportional-Integral kontrolcü. P ve I terimlerinin birleşimiyle steady-state hatası sıfıra indirgenir.
PWM	Pulse Width Modulation. Sabit frekansta, değişken duty cycle ile motor hızı kontrolü.
PVGIS	Photovoltaic Geographical Information System. Avrupa Komisyonu’nun güneş radyasyon veritabanı (TMY veri sağlar).

Septum	Burada LDR'ler arasındaki gölgelik bölme. Güneş açısına göre farklı LDR'leri gölgeler.
SIL	Software-in-the-Loop. Gömülü yazılımın bilgisayar simülasyonunda test edilmesi.
Stateflow	MATLAB/Simulink'in FSM modelleme aracı.
Stop Mode	STM32'nin düşük güç modlarından biri. RAM korunur, CPU durur.
TMY	Typical Meteorological Year. Tipik meteorolojik yıl verisi. Uzun dönemli ortalamalardan türetilmiş 8760 saatlik veri seti.
UART	Universal Asynchronous Receiver-Transmitter. Asenkron seri haberleşme protokolü.
Watchdog	Yazılımın çökme veya kilitlenme durumunda donanımı sıfırlayan zamanlayıcı.
Worm gear	Sonsuz dişli. Tek yönde tahrik edilebilen, ters yönde kilitlenen mekanik aktarım.